

SUMMER: Recursive Zero-Knowledge Proofs for Scalable RNN Training

Yuang Li
Rutgers University
Piscataway, USA
yl1407@rutgers.edu

Xiong Fan
Cysic, Inc.
China
leofanxiong@gmail.com

Abstract—Zero-knowledge proofs of training (zkPoT) enable a prover to certify that a model was trained on a committed dataset under a prescribed algorithm without revealing the model or data. Proving recurrent neural network (RNN) training is challenging due to hidden-state recurrence and cross-step weight sharing, which require proofs to enforce recurrence, gradients, and nonlinear activations across time.

We present SUMMER (SUMcheck and MERkle tree), a recursive zkPoT for scalable RNNs. SUMMER generates sumcheck-based proofs that BackPropagation Through Time (BPTT) was computed correctly over a quantized finite field, while nonlinearities such as tanh and softmax are validated by lookup arguments. Per-step commitments and proofs are folded with Merkle trees, yielding a final commitment and a succinct proof whose size and verification time are independent of the number of iterations.

SUMMER offers (i) a novel end-to-end zkPoT for RNN training, including forward, backward, and parameter updates; (ii) the use of LogUp for nonlinear operations with a batched interface; and (iii) efficient recursive composition of lookup and sumcheck proofs. On a Mini-Char-RNN with 12 million parameters, the prover runs in 70.1 seconds per iteration, $8.5\times$ faster and $11.6\times$ more memory efficient than the IVC baseline, with 165 kilobytes proofs verified in 20 milliseconds.

Index Terms—cryptography, zero knowledge proof, verifiable machine learning

1. Introduction

Artificial intelligence has advanced rapidly with large language models (LLMs), which power applications from chatbots to code generation. These models build on earlier architectures such as recurrent neural networks (RNNs), the first to capture sequential dependencies by processing inputs step by step. While modern LLMs use Transformers for efficiency and scale, RNNs remain the fundamental precursor to today’s generative AI.

Yet both RNNs and LLMs face a trust problem: users cannot easily verify how models were trained, whether data sources were valid, or if outputs are faithful to the claimed computation. This black-box nature raises concerns around integrity, reproducibility, and accountability, especially in high-stakes domains like medicine or finance.

Zero-knowledge proofs (ZKPs) offer a path forward. Originally developed to prove statements without revealing secrets, ZKPs now power applications from

blockchain scalability to privacy-preserving authentication. Applied to machine learning, they can prove the correctness of training and inference without exposing data or proprietary weights. For RNNs, ZKPs can certify forward passes, activations, and gradient updates, and—through recursion—yield succinct proofs that cover entire training runs. In this way, ZKPs help bridge the trust gap, enabling verifiable AI systems that are not only powerful but also provably reliable.

While much of the current research on verifiable AI focuses on proof of inference, the challenge of proof of training is both harder and more critical. Training involves multiple iterative steps: forward passes through the network, non-linear activations, loss evaluations, and gradient-based updates to model parameters. Each of these stages introduces additional complexity that must be faithfully proven, and when repeated across thousands of batches, the proof system must scale without substantial increase in size. Verifying training is essential because it establishes trust in the origin of the model itself. Without proof of training, one can only check outputs from a black box; with it, we can ensure that the entire learning process was conducted honestly, on the claimed data, and without hidden manipulations. This shift from verifying predictions to verifying the full lifecycle of model creation represents a crucial step toward trustworthy AI.

1.1. Related Works

With performance improving in ZK proof generation, the community has witnessed an enormous amount of work on verifiable AI. We start from the more related verifiable training line of work. In zkDL [1], the authors introduced how to handle the ReLU function and more generally the non-linear activation function. Their system relies on Pedersen commitment scheme and applies the GKR proof system layer-by-layer, showing the feasibility result but leaving scalability issues open for large models. zkPoT [2] expands the scope of zkDL work, but its construction cannot be naturally extended to deep neural networks due to fundamental limitations in handling complex nonlinear functions and iterative updates.

The second aspect of verifiable AI is about integrity in inference. zkCNN [3] proposed one of the first approaches to verifiable convolutional neural networks, introducing specialized sumcheck protocols tailored to two-dimensional convolutions as well as to FFT operations that arise in CNN spectral domain acceleration. After that, several works [4]–[6] improve on several sectors of verifiable

inference, such as optimized Boolean representation, advanced constraint representation, and expansion to large-scale models. The most recent work [7] represents the next step forward, in delivering a system explicitly designed to prove the correctness of GPT-scale architectures.

The most related work that we should consider is Kaizen [8], where they designed and implemented a *zkPoT* framework targeted at deep neural networks (DNNs). In their construction, they used a two-step process: first, a GKR system is deployed to prove training result of each iteration, and then these step-proofs are recursively composed to obtain a succinct proof¹. However, as their construction heavily relies on bit-decomposition for non-linear computation, such as exponentiation, ReLU and Softmax, which incurs significant increase in the number of constraints and thus the proving time. Kaizen currently provides support for ReLU and Softmax; common functions like sigmoid and tanh can be implemented via the same exponentiation lookup gadget, but not wired into the public benchmarks. On the other hand, Kaizen does not provide a true end-to-end training proof—its design repeats per-iteration step proofs within a recursive wrapper.

Compared with Kaizen, SUMMER takes a different path toward scalability. Instead of bit-decomposition-based nonlinear lookups, we adopt a LogUp-style [9] batched lookup argument, which eliminates the requirement for large Boolean tables and substantially reduces constraint size and memory footprint. Furthermore, SUMMER introduces a provable parameter-update mechanism that links consecutive training iterations, thereby enabling a genuine end-to-end proof of training rather than isolated iteration proofs. While Kaizen primarily targets CNNs and achieves proof aggregation through Orion-style [10] commitments, SUMMER focuses on sequence models such as RNNs, combining structured lookups, efficient recursion, and quantized computation to achieve constant-size verification and sublinear prover overhead relative to raw training time.

1.2. Our Contributions

Our system constructs an end-to-end prover and verifier pipeline that encompasses every stage of the training process. This includes the forward pass, the handling of non-linear activations, the calculation of loss, and the backpropagation steps required to update gradients and weights. By capturing the complete training loop, SUMMER makes it possible to prove correctness and integrity of the training procedure itself, ensuring that the final model parameters were derived honestly.

Our work introduces several novel contributions to the design of zero-knowledge proofs of training. First of all, we present the a novel ZKP system for RNN training, not merely inference. SUMMER builds a unified prover/verifier pipeline that covers the entire training loop, including the forward pass, nonlinear activations, loss computation, gradient derivation, and parameter updates, thereby certifying that the final model parameters arise from an honest execution of BackPropagation Through Time (BPTT). In the implementation, we apply the LogUp

lookup argument [9] to training, proving nonlinearities like tanh and Softmax/exp, and introduce a batching interface (ExpBatch) with sumcheck checks for efficient activations. We also design a recursive composition framework that folds iterations into a single succinct proof, binding all lookup claims and updates while keeping proof size and verification time constant. Finally, we evaluate SUMMER on RNNs such as Mini-Char-RNN [11], showing scalability: for a 12M-parameter model, proofs take 70.1s per iteration, are 165KB verified in 20ms, and improve proving speed $8.5\times$ and memory use $11.6\times$ over the state-of-the-art [12].

2. Preliminaries

Notation. In this work, we use λ to denote the security parameter and negl to denote a negligible function. We use $\mathbb{F}$ to denote a finite field. "PPT" stands for probabilistic polynomial time. We use bold lowercase letters (e.g., $\mathbf{v}, \mathbf{w}$) to denote vectors and bold uppercase letters (e.g., $\mathbf{A}, \mathbf{B}$) to denote matrices. For a vector $\mathbf{v} \in \mathbb{F}^\ell$, we use lowercase letters to refer to its components; that is, we write $\mathbf{v} = (v_1, \dots, v_\ell)$.

2.1. RNN and Training

Recurrent Neural Networks (RNNs) [13] are deep learning models that capture the dynamics of sequences via recurrent connections, which can be thought of as cycles in the network of nodes.

An RNN operates on an input space and an internal state space, which maintains a memory trace of previously processed inputs. The state space enables the representation of sequentially extended dependencies over unspecified intervals according to

$$\mathbf{y}^{(t)} = G(\mathbf{s}^{(t)}), \quad \mathbf{s}^{(t)} = F(\mathbf{s}^{(t-1)}, \mathbf{x}^{(t)}).$$

We consider a standard Recurrent Neural Network (RNN) with an input layer, one hidden layer, and an output layer. For notational convenience, we assume that the input at time step t is already an *embedded* vector: let $\mathbf{x}^{(t)} \in \mathbb{R}^d$ denote the token embedding of token $w^{(t)}$, obtained via an embedding matrix $\mathbf{E} \in \mathbb{R}^{|\mathcal{V}| \times d}$. For a length- T sequence we stack the inputs as $\mathbf{X} = [\mathbf{x}^{(1)}; \dots; \mathbf{x}^{(T)}] \in \mathbb{R}^{T \times d}$.

2.1.1. Forward Pass. The model directly consumes the input vector $\mathbf{x}^{(t)}$ (no separate embedding step). Let $\mathbf{X}$ denote the stacked input matrix whose t -th row is $\mathbf{x}^{(t)}$. Using hidden state $\mathbf{h}^{(t)}$, weight matrices $\mathbf{W}_h, \mathbf{W}_e, \mathbf{W}_y$ for hidden, input, and output layers, respectively, and bias terms b_1, b_2 , the forward update is

$$\begin{aligned} \mathbf{h}^{(t)} &= \text{Act}(\mathbf{W}_h \mathbf{h}^{(t-1)} + \mathbf{W}_e \mathbf{x}^{(t)} + b_1), \\ \hat{\mathbf{y}}^{(t)} &= \text{softmax}(\mathbf{W}_y \mathbf{h}^{(t)} + b_2). \end{aligned}$$

Here, Act denotes the element-wise activation function, which introduces non-linearity into the model, enabling it to capture complex patterns. Common choices for activation functions include $\text{ReLU}(x) = \max(x, 0)$, $\text{sigmoid}(x) = 1/(1 + e^{-x})$, and $\text{tanh}(x) = (e^{2x} - 1)/(e^{2x} + 1)$. For the output layer, we follow standard practice and apply the $\text{Softmax}(x_i) = (e^{x_i})/(\sum_j e^{x_j})$ to

1. <https://github.com/zkPoTs/kaizen>

normalize the logits into a probabilistic distribution over the possible output classes.

Let K be the number of output classes, and $y_k^{(t)}$ be the one-hot encoded ground-truth label for class k at time step t , with $\hat{y}_k^{(t)}$ denoting the predicted probability corresponding. The loss at time step t is computed using the cross-entropy loss: $\mathcal{L}^{(t)} = -\sum_{k=1}^K y_k^{(t)} \log(\hat{y}_k^{(t)})$, while the total loss over T steps is: $\mathcal{L} = \frac{1}{T} \sum_{t=1}^T \mathcal{L}^{(t)}$.

2.1.2. Backward Pass. Due to the recursive nature of RNNs, gradients must be computed through time. This is achieved via BackPropagation Through Time (BPTT) [14]. Let $\delta^{(t)} = \hat{\mathbf{y}}^{(t)} - \mathbf{y}^{(t)}$ denote the output error at the time step t . Gradients for the output layer are

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_y} = \delta^{(t)} (\mathbf{h}^{(t)})^\top, \quad \frac{\partial \mathcal{L}}{\partial b_2} = \delta^{(t)}.$$

The gradient is backpropagated through the hidden layer using the chain rule:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{h}^{(t)}} = \mathbf{W}_y^\top \delta^{(t)}, \quad \frac{\partial \mathcal{L}}{\partial \mathbf{a}^{(t)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{h}^{(t)}} \circ \text{Act}'(\mathbf{a}^{(t)}),$$

where the pre-activation is $\mathbf{a}^{(t)} = \mathbf{W}_h \mathbf{h}^{(t-1)} + \mathbf{W}_e \mathbf{x}^{(t)} + b_1$.

Gradients for the parameters of the hidden layer follow:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_h} = \left(\frac{\partial \mathcal{L}}{\partial \mathbf{a}^{(t)}} \right) (\mathbf{h}^{(t-1)})^\top, \quad \frac{\partial \mathcal{L}}{\partial \mathbf{W}_e} = \left(\frac{\partial \mathcal{L}}{\partial \mathbf{a}^{(t)}} \right) (\mathbf{x}^{(t)})^\top, \\ \frac{\partial \mathcal{L}}{\partial b_1} = \frac{\partial \mathcal{L}}{\partial \mathbf{a}^{(t)}}.$$

Finally, gradients to the previous hidden state are:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{h}^{(t-1)}} = \mathbf{W}_h^\top \left(\frac{\partial \mathcal{L}}{\partial \mathbf{a}^{(t)}} \right).$$

2.1.3. Parameter Update. After computing the gradients of the loss with respect to each trainable parameter using backpropagation through time (BPTT), the parameters are updated via (stochastic) gradient descent:

$$\mathbf{W}_y \leftarrow \mathbf{W}_y - \eta \cdot \frac{\partial \mathcal{L}}{\partial \mathbf{W}_y}, \quad b_2 \leftarrow b_2 - \eta \cdot \frac{\partial \mathcal{L}}{\partial b_2}, \\ \mathbf{W}_h \leftarrow \mathbf{W}_h - \eta \cdot \frac{\partial \mathcal{L}}{\partial \mathbf{W}_h}, \quad \mathbf{W}_e \leftarrow \mathbf{W}_e - \eta \cdot \frac{\partial \mathcal{L}}{\partial \mathbf{W}_e}, \\ b_1 \leftarrow b_1 - \eta \cdot \frac{\partial \mathcal{L}}{\partial b_1}.$$

where $\eta > 0$ is the *learning rate* that controls the step size, a hyperparameter that controls the size of the update step.

In practice, adaptive optimizers like Adam, RMSProp, or Adagrad [15]–[17] are often used to adjust the effective learning rate per-parameter; gradient clipping [18] is also commonly applied in RNNs to mitigate exploding gradients. The detailed algorithm are presented in Algorithm 1 of Appendix A.1.

2.2. Proofs, Arguments, and Zero-Knowledge

2.2.1. Interactive Proof. An interactive proof allows a prover $\mathcal{P}$ to convince a verifier $\mathcal{V}$ of a statement’s validity through multiple rounds, with $\mathcal{V}$ ’s queries depending on $\mathcal{P}$ ’s prior answers. Such a protocol satisfies completeness if an honest proof is always accepted, and soundness if a false proof convinces the verifier only with negligible probability. Knowledge soundness further ensures a prover cannot succeed without holding a valid witness. When soundness holds only against computationally bounded provers, the system is called an argument. A proof/argument is zero-knowledge if it reveals nothing about the witness beyond the truth of the statement, and succinct if $\mathcal{V}$ ’s runtime and communication are only $\text{poly}(\lambda, |x|, \log |w|)$.

A protocol is public-coin if the verifier’s messages are independent of the prover’s. Via the Fiat–Shamir transformation [19], such protocols can be converted into non-interactive proofs in the random oracle model.

2.2.2. Polynomial Commitment Schemes. A polynomial commitment scheme (PCS) lets a prover $\mathcal{P}$ commit to a polynomial and later prove its evaluation at chosen points. A PCS is hiding if the commitment leaks no information about the polynomial, and binding if a prover cannot open one commitment to different values at the same point. Knowledge soundness ensures that any valid evaluation proof implies knowledge of the committed polynomial, while zero-knowledge guarantees the proof reveals nothing beyond the claimed evaluation. Formal definitions appear in Appendix A.2.

2.2.3. Incrementally Verifiable Computation (IVC). Incrementally Verifiable Computation (IVC) [20] certifies long computations by proving each step while keeping verification cost independent of the total number of steps. The computation is modeled as $z_{i+1} = F(z_i, \omega_i)$, with z_i as public state and ω_i as the private witness.

Each step is wrapped in an augment function [12] F_A , which, given $(i+1, z_0, z_i, \omega_i, \pi_i, s_i)$, outputs $(\text{verdict}_i, z_{i+1}, s_{i+1})$. The in-circuit verifier $\mathcal{V}_0$ checks the prior proof, enforcing $\text{verdict}_i = 1$ before advancing. The accumulator s_{i+1} stores minimal data (e.g., counters, digests) needed for recursion.

At each round, the prover evaluates F_A in-circuit and produces a succinct proof Π_{i+1} showing the prior proof was valid, the state transitioned correctly, and the accumulator updated consistently. The external verifier need only check the final (or any chosen) succinct proof, plus a constant number of deferred openings, rather than re-verifying all steps.

2.3. GKR-based Proof Systems

2.3.1. Multilinear Extension. Let $f : \{0, 1\}^\ell \rightarrow \mathbb{F}$ be a function. The *multilinear extension* of f is defined as the unique polynomial $\tilde{f} : \mathbb{F}^\ell \rightarrow \mathbb{F}$ such that $\tilde{f}(x_1, \dots, x_\ell) = f(x_1, \dots, x_\ell)$ for all $x_1, \dots, x_\ell \in \{0, 1\}$.

$\tilde{f}$ can be expressed as:

$$\tilde{f}(x_1, \dots, x_\ell) = \sum_{b \in \{0, 1\}^\ell} L_b(x_1, \dots, x_\ell) \cdot f(b).$$

Here, L_b is called the *Lagrange kernel* and is defined as

$$L_b(x_1, \dots, x_\ell) = \prod_{i=1}^{\ell} (b_i x_i + (1 - b_i)(1 - x_i)),$$

where b_i is the i -th bit of b . The crucial property of the Lagrange kernel is that if $b, b' \in \{0, 1\}^\ell$, $L_b(b') = 1$ if and only if $b' = b$, otherwise $L_b(b') = 0$.

2.3.2. Merkle Tree. Merkle tree [21] is a cryptographic primitive that is used to commit to a vector and open it at an index with logarithmic proof size and verification time. It consists of three algorithms:

- $\text{root} \leftarrow \text{MT.Commit}(x; r)$: on input a vector x and randomness r , the commitment algorithm returns the root of the Merkle tree as a commitment to x .
- $(x_i, \pi_i) \leftarrow \text{MT.Open}(i, x; r)$: on input a vector x , an opening index i and randomness r , the opening algorithm returns a vector element x_i and a proof π_i .
- $(1, 0) \leftarrow \text{MT.Verify}(\text{root}, i, x_i, \pi_i)$: on input a commitment root, an index i , an element x_i , and a proof π_i , the verification algorithm returns 1 if the proof is valid, otherwise returns 0.

2.3.3. Sumcheck Protocol. The *sumcheck protocol* [22] is an interactive proof system that enables a verifier to check whether the sum of a multivariate polynomial over an exponentially large domain is equal to the claimed target value, without evaluating all terms directly.

Given an ℓ -variate polynomial $g : \mathbb{F}^\ell \rightarrow \mathbb{F}$ over a finite field $\mathbb{F}$, define the total sum

$$T := \sum_{b_1 \in \{0,1\}} \sum_{b_2 \in \{0,1\}} \cdots \sum_{b_\ell \in \{0,1\}} g(b_1, \dots, b_\ell),$$

which can equivalently be written as $T := \sum_{x \in \{0,1\}^\ell} g(x)$.

The protocol proceeds in ℓ rounds. In each round, the prover sends a univariate polynomial, and the verifier responds with a random challenge. At the end of the protocol, the verifier checks a single evaluation of g at a random point $r \in \mathbb{F}^\ell$. For completeness, we describe the detailed protocol in Protocol A.3 of Appendix A.3.

2.3.4. GKR Protocol. Goldwasser, Kalai, and Rothblum introduced the celebrated GKR [23] protocol, a highly efficient interactive proof system to verify computations represented by layered arithmetic circuits. Let $\mathcal{C} : \mathbb{F}^n \rightarrow \mathbb{F}^k$ be a layered arithmetic circuit with d layers over a finite field $\mathbb{F}$. Let layer 0 be the output layer and layer d the input layer. The prover $\mathcal{P}$ wants to convince a verifier $\mathcal{V}$ that $\text{out} = \mathcal{C}(\text{input})$ where in is the input from $\mathcal{V}$ and out is the corresponding output computed by the circuit.

Let S_i be the number of gates in the i -th layer and let $s_i = \lceil \log S_i \rceil$. We define a function $V_i : \{0, 1\}^{s_i} \rightarrow \mathbb{F}$ that takes binary inputs $b \in \{0, 1\}^{s_i}$ and outputs the output of gate b in layer i , where b is called the *gate label*. Each gate in layer i is wired to two gates in layer $i+1$ through fixed wiring patterns, which specify the inputs used for each gate's computation. These wires can be described using *wire predicates*, which are Boolean functions that indicate whether a tuple of gate indices corresponds to a valid connection. The multiplicative wiring predicate $\text{mult}_i(u, v, w) : \{0, 1\}^{s_i+2s_{i+1}} \rightarrow \{0, 1\}$ outputs 1 if

gate u in layer i computes the product of gates v and w from layer $i+1$. Similarly, the additive wiring predicate $\text{add}_i(u, v, w)$ captures the addition gates.

We then extend V_i to its multilinear extension $\tilde{V}_i : \mathbb{F}^{s_i} \rightarrow \mathbb{F}$. Then $\tilde{V}_i$ can be computed as:

$$\begin{aligned} \tilde{V}_i(u) = & \sum_{v, w \in \{0,1\}^{s_{i+1}}} (\widetilde{\text{add}}_i(u, v, w) \cdot (\tilde{V}_{i+1}(v) + \tilde{V}_{i+1}(w)) \\ & + (\widetilde{\text{mult}}_i(u, v, w) \cdot (\tilde{V}_{i+1}(v) \cdot \tilde{V}_{i+1}(w))) \end{aligned} \quad (1)$$

for any $u \in \{0, 1\}^{s_i}$.

Once $\mathcal{V}$ receives the claimed output from $\mathcal{P}$, $\mathcal{V}$ samples a random point r_0 and evaluates $\tilde{V}_0(r_0)$. In the first round, $\mathcal{V}$ and $\mathcal{P}$ run the sumcheck protocol on Equation 1. At the end of the protocol $\mathcal{V}$ receives two evaluations $\tilde{V}_1(r_{1,0}), \tilde{V}_1(r_{1,1})$ of $\tilde{V}_1$ at two distinct random point $r_{1,0}, r_{1,1}$. $\mathcal{V}$ randomly samples $\alpha, \beta \in \mathbb{F}$ and computes $\alpha \tilde{V}_1(r_{1,0}) + \beta \tilde{V}_1(r_{1,1})$. Then $\mathcal{V}$ and $\mathcal{P}$ run sumcheck protocol on

$$\begin{aligned} \alpha \tilde{V}_1(r_{1,0}) + \beta \tilde{V}_1(r_{1,1}) = & \sum_{x, y \in \{0,1\}^{s_2}} \\ & \left(\alpha \cdot \widetilde{\text{mult}}_1(r_{1,0}, x, y) + \beta \cdot \widetilde{\text{mult}}_1(r_{1,1}, x, y) \right) \cdot \tilde{V}_2(x) \cdot \tilde{V}_2(y) + \\ & \left(\alpha \cdot \widetilde{\text{add}}_1(r_{1,0}, x, y) + \beta \cdot \widetilde{\text{add}}_1(r_{1,1}, x, y) \right) \cdot (\tilde{V}_2(x) + \tilde{V}_2(y)). \end{aligned}$$

At the end of the protocol, $\mathcal{V}$ receives two claims about $\tilde{V}_2$, computes their random linear combination, and proceeds to an above layer recursively until the input layer. At input layer d , $\mathcal{V}$ can check the evaluations of $\tilde{V}_d$ by querying the oracle. The detailed protocol is shown in Protocol A.4 of Appendix A.4.

2.4. Lookup Arguments

Lookup arguments are a powerful cryptographic primitive for proving statements about every element of a *private* vector $\mathbf{v}$ is contained in a *public*, usually pre-computed table T . This is particularly useful in contexts where a computation involves repeated accesses to a set of predetermined values. Formally, the statement proven in a lookup argument is defined as:

Definition 1 (Lookup Argument [24]). *Given a commitment cm_a and a public set T of N field elements, represented as vector $\mathbf{t} = (t_0, \dots, t_{N-1}) \in \mathbb{F}^N$ to which the verifier has been provided a commitment cm_t , the prover knows an opening $\mathbf{a} = (a_0, \dots, a_{m-1}) \in \mathbb{F}^m$ of cm_a such that for each $i = 0, \dots, m-1$, there is a $j \in \{0, \dots, N-1\}$ such that $a_i = t_j$.*

Modern ZKP systems rely on lookup arguments to capture non-linear relations compactly. In RNN trainings, numerous non-linear operations, such as activation functions (e.g., ReLU, tanh, softmax), range check, and comparison relations are naturally encoded as lookups.

We highlight two complementary lines that are widely used in practice. The *Lasso-style* [24], [25] approach decomposes the *structured* public table T into much smaller subtables, so that each query $\mathbf{v}[i] \in T$ reduces to consistent sub-queries. The other approach, known as *LogUp-style* [9], [26], utilizes a logarithmic derivative to

certify membership through a single rational identity in a random challenge c :

$$\sum_{i=0}^{m-1} \frac{1}{c - \mathbf{v}[i]} = \sum_{j=0}^{n-1} \frac{\text{mult}[j]}{c - T[j]}$$

where mult denotes the multiplicity that counts the number of times each element $T[j]$ is included in the queries.

In our work, we adopt *TaSSLe* [27], which synthesizes Lasso’s decomposability with a LogUp-style rational sumcheck, thereby significantly reducing the commitment costs. We pre-compute and commit shared activation and auxiliary tables and batch all per-time-step queries across the unrolled sequence, achieving linear prover cost in the number of lookups and few commitment openings for the verifier. For technical details of TaSSLe, please refer to [27].

3. Technical Overview

3.1. Quantization

Neural training uses floating-point arithmetic that is incompatible with arithmetic circuits in zero-knowledge proof. While several schemes exist (e.g., uniform affine quantization [3], [7]), SUMMER adopts a power-of-two fixed point-encoding over a large prime field to keep constraints simple, avoid per-tensor zero-points, and eliminate bit-decomposition during rescaling.

Let $\mathbb{F}_{p^2}$ denote a quadratic extension prime field with a Mersenne prime p . An element is represented as $a + bu$ with $u^2 = v$ a quadratic non-residue. For the training computation itself, we embed all quantized values in the base subfield by setting the imaginary part to zero, i.e. ($b = 0$), so arithmetic behaves exactly like $\mathbb{F}_p$ while the extension field remains available for FFT-friendly commitments and recursive composition.

3.1.1. Quantization and Dequantization. We quantize real numbers with Q fractional bits by scaling with 2^Q and rounding *toward zero*: $s = \text{trunc}(r \cdot 2^Q) \in \mathbb{Z}$. Before embedding, we apply symmetric clipping to avoid wrap-around:

$$s \leftarrow \text{clip}_q(s) \in [- (2^{q-1}), 2^{q-1} - 1],$$

and store $\text{quantize}(r) = s \bmod p \in \mathbb{F}_p$. Positive codes are kept as-is; negative codes $s < 0$ are represented as $p + s$ (the high half of $\mathbb{F}_p$). Decoding lifts $a \in \mathbb{F}_p$ to its signed representative $\tilde{s} \in (-\frac{p}{2}, \frac{p}{2}]$ and returns $\text{dequantize}(a, \text{depth}) = \tilde{s} / (2^Q)^{\text{depth}}$. We use no per-tensor zero-points; symmetric clipping ensures sign disambiguation and prevents rare overflows without changing the arithmetic in the base subfield.

3.1.2. Integer arithmetic after quantization. Addition and subtraction operate directly in $\mathbb{F}_p$ and preserve the Q -fraction scale. For multiplication, if s_a, s_b are the encoded integers, the product $z = s_a s_b$ has scale $2Q$; we downscale by a power of two using shift (Euclidean division by 2^Q) and take the returned quotient as the rescaled result at Q fractional bits.

3.2. Sumcheck-based Proofs for Backpropagation Through Time (PBPTT)

Our PBPTT protocol attests to the correctness of the RNN training by separating the computation into two parts: (i) *linear operations*, including matrix-matrix/vector products, Hadamard products, and summations; (ii) *non-linear maps*, including activations such as $\exp$ ($\tanh$, Softmax) and $\max$ (ReLU). For composability and verifier efficiency, we use a sumcheck-based approach throughout. All linear operations are reduced to matrix multiplication and proved with a time optimal matrix multiplication sumcheck protocol, and all nonlinearities are verified via lookup arguments over fixed point tables produced in the quantization step.

3.2.1. Linear Operations. We reduce every linear subroutine to matrix multiplication and invoke Thaler’s time optimal sumcheck protocol for matrix multiplication [28]. We denote this subprotocol by $\text{Sum}_{\text{MatMul}}$, and apply it uniformly to rectangular shapes $\mathbf{M}_1 \in \mathbb{F}^{p \times t}$, $\mathbf{M}_2 \in \mathbb{F}^{q \times t}$, and $\mathbf{M}_1 \mathbf{M}_2^\top \in \mathbb{F}^{p \times q}$. In particular, for $n \times n$ inputs the prover runs in $\tilde{O}(n^2)$ time.

Matrix–vector multiplication is treated as a special case of this canonical form.

Matrix vector: $\mathbf{y} = \mathbf{M}\mathbf{x}$ ($\mathbf{M} \in \mathbb{F}^{p \times q}$, $\mathbf{x} \in \mathbb{F}^q$, $\mathbf{y} \in \mathbb{F}^p$)

Hadamard multiplication is represented as multiplication by a diagonal operator.

$$\text{Hadamard: } \mathbf{x} \circ \mathbf{y} = \text{diag}(\mathbf{x}) \mathbf{y} \\ (\mathbf{x}, \mathbf{y} \in \mathbb{F}^q, \text{diag}(\mathbf{x}) \in \mathbb{F}^{q \times q}, \mathbf{x} \circ \mathbf{y} \in \mathbb{F}^q)$$

Summation is handled by stacking and concatenation so that the total becomes a single multiplication.

$$\text{Summation: } \sum_{t=1}^T \mathbf{M}^{(t)} \mathbf{x}^{(t)} = [\mathbf{M}^{(1)} \dots \mathbf{M}^{(T)}] \begin{bmatrix} \mathbf{x}^{(1)} \\ \vdots \\ \mathbf{x}^{(T)} \end{bmatrix} \\ (\mathbf{M}^{(t)} \in \mathbb{F}^{p \times q}, \mathbf{x}^{(t)} \in \mathbb{F}^q, \sum_t \mathbf{M}^{(t)} \mathbf{x}^{(t)} \in \mathbb{F}^p)$$

Under these reductions, $\text{Sum}_{\text{MatMul}}$ proves the matrix multiplication claims that arise from all linear layers in the forward and backward passes. Henceforth, whenever such a linear operation appears, we simply invoke $\text{Sum}_{\text{MatMul}}$ on its canonical reduction.

3.2.2. Non-linear operations. Nonlinear computations in RNNs include $\max$ and exponentiations. These are costly to arithmetize inside a zero-knowledge arithmetic circuit. As an example, one softmax requires exponentiations and a normalization, which leads to many constraints if expanded directly. We avoid this expansion by using lookup arguments into public pre-committed tables under fixed-point quantization. For activations we certify membership of input–output pairs in the corresponding table.

$$(x, y) \in T_{\text{ReLU}} \quad \text{encodes } y = \max\{x, 0\} \\ (z, u) \in T_{\text{exp}} \quad \text{encodes } u = \exp(z) \\ p_j = \frac{u_j}{\sum_\ell u_\ell} \quad \text{with } (z_j, u_j) \in T_{\text{exp}}$$

We instantiate the lookup argument with a LogUp-style scheme, TaSSLE [27]. Crucially, the resulting batched lookup proof composes via sumcheck, so all nonlinear claims in an iteration are aggregated together and then *merged* with the sumcheck proofs of linear operations. After transcript folding, the verifier checks only a constant number of polynomial openings, and the same lookup commitments are reused across time steps.

3.3. Recursive Composition of Proofs

In each iteration, the PBPTT protocol emits many artifacts, including commitment opening proofs for model parameters and lookup tables, sumcheck transcripts for linear operations, and Fiat–Shamir challenges and digests. As the number of iterations grows, the verification time, proof size, and prover memory overhead increase linearly. We address this with an incrementally verifiable computation (IVC) scheme.

We adopt the multivariate commitment aggregation method of Abbaszadeh et al. [8]. Concretely, we commit to an aggregated codeword $\mathbf{M}^* = \sum_i \alpha_i \mathbf{M}_i$ and check the linear relation at $\Theta(\lambda)$ randomly sampled codeword positions. Then we open Merkle-tree paths for $\mathbf{M}^*$ and each $\mathbf{M}_i$ to verify $\mathbf{M}^*[r, c] = \sum_i \alpha_i \mathbf{M}_i[r, c]$ using the same sumcheck framework, which composes cleanly with our other sumcheck proofs and the recursive composition.

In each iteration, the prover generates a set of evaluation claims from $\text{Sum}_{\text{MatMul}}$ and simple vector operations, written as $\{(\sigma_j, r_j, v_j)\}_{j=1}^q$ where σ_j commits to a polynomial, r_j is the verifier challenge, and v_j is the claimed evaluation. We compress these claims by a sumcheck-based evaluation reduction followed by random linear combination, and we aggregate the corresponding commitments using the method of Abbaszadeh et al.. Across iterations we append only the aggregated pair $(\sigma_{\text{agg}}, v_{\text{agg}})$ to the recursive accumulator. At the end of IVC we sample fresh challenges and apply one final evaluation reduction so that all remaining claims are bound to a constant number of points, then we perform a constant number of batched openings. This yields a constant number of commitment openings for all matrix and vector evaluations, while the intermediate reductions use the same sumcheck framework as the rest of PBPTT.

For security and efficiency, we use a k -ary recursion tree in the sense of Bitansky et al. [29]. Leaf nodes are per-iteration PBPTT step proofs together with their matrix/vector evaluation claims and lookup commitments. At each internal node, the parent invokes the verifier circuit on its k child proofs and, once they pass, it invokes the commitment aggregation scheme together with evaluation reduction to combine the k children into a single parent proof and commitment. This procedure repeats level by level until the root is formed (depth $\lceil \log_k I \rceil$), where I is the number of iterations. At the root, the external verifier checks one recursive proof and a constant number of aggregated openings. The design reduces the number of IVC folds, keeps verifier time and proof size logarithmic in I , and preserves soundness via the IVC properties, commitment binding, and the sumcheck-based subprotocols.

4. Proofs for Backpropagation Through Time for Vanilla RNN

Recall a standard Recurrent Neural Network (RNN) with an input layer, a single hidden layer, and an output layer. At iteration i , the input-label sequences are $\{\mathbf{x}_i^{(t)}\}_{t=1}^T$ and $\{\mathbf{y}_i^{(t)}\}_{t=1}^T$. At each time step t ,

$$\begin{aligned} \mathbf{h}_i^{(t)} &= \text{Act}(\mathbf{W}_x \mathbf{x}_i^{(t)} + \mathbf{W}_h \mathbf{h}_i^{(t-1)} + b_1) \\ \hat{\mathbf{y}}_i^{(t)} &= \text{softmax}(\mathbf{W}_y \mathbf{h}_i^{(t)} + b_2) \end{aligned}$$

where Act denotes the activation function. The model is trained by minimizing cross-entropy loss per step $\mathcal{L}_i^{(t)}$, and gradients are computed via Backpropagation Through Time (BPTT). The weights $\mathbf{W}_y, \mathbf{W}_h, \mathbf{W}_x$ and the biases b_1, b_2 are updated by gradient descent with the learning rate η .

Our Proof of BPTT (PBPTT) protocol utilizes sumcheck protocol to attest the correctness of the training process and the gradient-based update equations. Concretely, the prover $\mathcal{P}$ employs a polynomial commitment scheme (PCS) to give verifier $\mathcal{V}$ oracle access to the inputs and outputs of the BPTT algorithm (Algorithm 1), together with auxiliary commitments supporting TaSSLE-based lookup arguments for non-linearities and range/compare relations.

We organize the PBPTT proofs into four stages, proceeding from output to input, with an initialization stage that fixes the committed instance.

Stage 0 (initialization). $\mathcal{P}$ executes Algorithm 1 locally for iteration i on inputs $(\{\mathbf{x}_i^{(t)}, \mathbf{y}_i^{(t)}\}_{t=1}^T, \theta^{(i)})$, recording the complete training trace (hidden states, logits, per-step gradients and updated parameters $\theta^{(i+1)}$). It encodes these arrays as low degree polynomials in the index domain (e.g., time $\times$ dimension) and invokes PCS to commit: (i) the *inputs* (previous parameters) $\text{cm}_\theta^{(i)}$, (ii) the *outputs* (current parameters) $\text{cm}_\theta^{(i+1)}$, and (iii) the TaSSLE *lookup witnesses*. $\mathcal{P}$ sends these commitments to $\mathcal{V}$.

Stage I (parameter-update). For each parameter block $\theta \in \{\mathbf{W}_y, \mathbf{W}_h, \mathbf{W}_x, b_1, b_2\}$, $\mathcal{V}$ samples a random challenge r_θ in the index domain of the multilinear extension of θ and requests from $\mathcal{P}$ the evaluation $\tilde{\theta}_{i+1, \theta}$. $\mathcal{V}$ and $\mathcal{P}$ then run a (sumcheck-based) GKR protocol to verify, for each θ ,

$$\tilde{\theta}_{i+1, \theta}(r_\theta) = \tilde{\theta}_{i, \theta}(r_\theta) - \eta \cdot \sum_{t=1}^T \tilde{G}_\theta^{(t)}(r_\theta),$$

where $\tilde{\theta}_{i, \theta}$ is the multilinear extension of the input parameters and $\tilde{G}_\theta^{(t)}$ is the contribution of the gradient per time step to θ . At the end of the protocol, $\mathcal{V}$ obtains the random evaluations $\tilde{\theta}_{i, \theta}(r_\theta)$ and $\{\tilde{G}_\theta^{(t)}(r_\theta)\}_{t=1}^T$ (or their batched sum), which will be consumed in later stages.

Stage II (backward pass). $\mathcal{V}$ samples fresh challenges in the backward trace domain and, by $\text{Sum}_{\text{MatMul}}$, certifying BPTT recurrences and per-time-step gradient identities that justify the projected terms used in Stage 1:

$$\begin{aligned} \nabla_{\mathbf{W}_{x,i}} &= \mathbf{a}_i'^{(t)}(\mathbf{x}_i^{(t)})^\top, & \nabla_{\mathbf{W}_{h,i}}^{(t)} &= \mathbf{a}_i'^{(t)}(\mathbf{h}_i^{(t-1)})^\top \\ \nabla_{b_{1,i}}^{(t)} &= \mathbf{a}_i'^{(t)}. \end{aligned}$$

At the end of these subprotocols, $\mathcal{V}$ obtains the random evaluations of the MLEs $\tilde{\mathbf{a}}_i^{(t)}$, $\tilde{\mathbf{x}}_i^{(t)}$ and $\tilde{\mathbf{h}}_i^{(t-1)}$ at the challenge points of the protocol. With $\tilde{\mathbf{a}}_i^{(t)}$, $\mathcal{V}$ and $\mathcal{P}$ check the nonlinear relation $\mathbf{a}_i^{(t)} = \text{Act}'(\mathbf{h}_i^{(t)})$ using the TaSSLE lookup argument against the witnesses committed in Stage 0.

$\mathcal{V}$ then continues to verify, still via $\text{Sum}_{\text{MatMul}}$:

$$\mathbf{h}_i^{(t)} = \begin{cases} \mathbf{W}_y^\top \delta_i^{(t)}, & t = T, \\ \mathbf{W}_y^\top \delta_i^{(t)} + \mathbf{W}_h^\top \mathbf{a}_i^{(t+1)}, & t < T, \end{cases}$$

and the remaining gradient identities

$$\nabla_{\mathbf{W}_{y,i}}^{(t)} = \delta_i^{(t)} (\mathbf{h}_i^{(t)})^\top, \quad \nabla_{b_{2,i}}^{(t)} = \delta_i^{(t)} \\ \delta_i^{(t)} = \hat{\mathbf{y}}_i^{(t)} - \mathbf{y}_i^{(t)}.$$

This stage yields the required random evaluations of $\delta_i^{(t)}$, $\mathbf{a}_i^{(t)}$, $\mathbf{h}_i^{(t)}$, consistently tying them to the projected gradient terms $\tilde{G}_\theta^{(t)}(r_\theta)$ obtained in Stage 1.

Stage III (forward pass). With fresh challenges in the forward trace domain, $\mathcal{V}$ and $\mathcal{P}$ verify the forward equations, checking linear relations via $\text{Sum}_{\text{MatMul}}$ and certifying non-linearities through TaSSLE lookups.

$$\begin{aligned} \hat{\mathbf{y}}_i^{(t)} &= \text{Softmax}(\mathbf{z}_i^{(t)}) \\ \mathbf{z}_i^{(t)} &= \mathbf{W}_{y,i-1} \mathbf{h}_i^{(t)} + b_{2,i-1} \\ \mathbf{h}_i^{(t)} &= \text{Act}(\mathbf{a}_i^{(t)}) \\ \mathbf{a}_i^{(t)} &= \mathbf{W}_{h,i-1} \mathbf{h}_i^{(t-1)} + \mathbf{W}_{x,i-1} \mathbf{x}_i^{(t)} + b_{1,i-1} \end{aligned}$$

At the end of this stage, $\mathcal{V}$ obtains random evaluations of $\mathbf{h}_i^{(t)}$, $\mathbf{z}_i^{(t)}$ and $\hat{\mathbf{y}}_i^{(t)}$ (and, when necessary, $\mathbf{a}_i^{(t)}$), which are cross-checked against Stage II to ensure end-to-end consistency.

Stage IV (evaluation reduction and openings). $\mathcal{V}$ and $\mathcal{P}$ fold all evaluations emitted across Stages I–III into a *constant* number of evaluations per committed matrix using random linear combinations, producing single-point challenges $r_{\text{in},*}$ for input and $r_{\text{out},*}$ for output. $\mathcal{P}$ then invokes the PCS opening algorithm for each relevant commitment (including lookup-witness commitments).

$$\begin{aligned} &\text{PCS.Open}(\text{cm}_{\mathbf{W}_{*,i-1}}, r_{\text{in}, \mathbf{W}_{*,i-1}}) \\ &\text{PCS.Open}(\text{cm}_{\mathbf{W}_{*,i}}, r_{\text{out}, \mathbf{W}_{*,i}}) \\ &\text{PCS.Open}(\text{cm}_{\mathbf{b}_{*,i-1}}, r_{\text{in}, \mathbf{b}_{*,i-1}}) \\ &\text{PCS.Open}(\text{cm}_{\mathbf{b}_{*,i}}, r_{\text{out}, \mathbf{b}_{*,i}}) \\ &\text{PCS.Open}(\text{cm}_{\mathbf{x}_i}, r_{\text{in}, \mathbf{x}_i}) \\ &\text{PCS.Open}(\text{cm}_{\mathbf{y}_i}, r_{\text{in}, \mathbf{y}_i}) \\ &\text{PCS.Open}(\text{cm}_{L_i}, r_{\text{in}, L_i}) \end{aligned}$$

$\mathcal{V}$ verifies these openings together with the $\text{Sum}_{\text{MatMul}}$ /GKR transcripts to accept or reject.

Theorem 1. *The PBPTT protocol (Protocol B) satisfies completeness, knowledge soundness, and zero-knowledge.*

Completeness follows from the correctness of GKR, $\text{Sum}_{\text{MatMul}}$, and LogUp, an honest prover always produces an accepting transcript. Knowledge soundness holds since any forged parameter or gradient would be accepted with negligible probability via the Schwartz-Zippel lemma and

the binding of PCS. Zero-knowledge follows by invoking the simulators of each component, yielding transcripts indistinguishable from real proofs. We show the formal details in Protocol B and provide the security proof in Appendix B.

Cost analysis. Fix iteration i . Let T be the number of time steps, $s_{\text{in},i}$ and $s_{\text{out},i}$ the sizes of the committed inputs and outputs, and q the bit-length of quantized field elements. Write $N_i = s_{\text{in},i} + s_{\text{out},i} + s_{\text{param}} + s_{\text{LogUp}}$ which captures the algebraic size of one PBPTT iteration, including inputs, outputs, parameters, and batched LogUp lookup witnesses. Since the time of PBPTT is dominated by commitments, we adopt Orion as the polynomial commitment scheme: its prover runs in $O(qN_i)$ field operations, while the verifier time and proof size are both $O(\log^2 qN_i)$. Evaluation reduction ensures only $O(1)$ openings per iteration.

5. Recursive Proof Composition

Each PBPTT iteration produces multiple subproofs, commitments, and hash digests: sumcheck transcripts for linear operations and LogUp-based lookups, and PCS opening proofs for model and lookup commitments, together with Fiat–Shamir challenges and transcript digests. These subproofs, if verified sequentially, would incur verification time, proof size, and memory overhead that grow linearly with the number of iterations. We address this with incrementally verifiable computation (IVC).

We rely on the IVC step relation from Section 2.2.3. For PBPTT, we instantiate the state with the model parameters: we treat the parameter commitment from iteration i , $\text{cm}_\theta^{(i)}$, as the input z_i to F_A . Given $(i+1, z_0, \text{cm}_\theta^{(i)}, \omega_i, \pi_i, s_i)$, F_A rechecks π_i *in circuit*, enforces one round of BPTT (forward, backward, and update $\theta^{(i+1)} = \theta^{(i)} - \eta \nabla \theta^{(i)}$) using $\text{Sum}_{\text{MatMul}}$ for linear operations and batched lookups for non-linear maps, compresses the resulting evaluation claims, and outputs the commitment of the new parameter $\text{cm}_\theta^{(i+1)}$ together with the updated accumulator s_{i+1} , all certified by a succinct proof.

Evaluation reduction and aggregated openings. We unify the treatment of sumcheck evaluations and PCS openings in one pipeline. After enforcing the current PBPTT constraints, the step collects evaluation claims $\{(\sigma_j, r_j, v_j)\}$ from $\text{Sum}_{\text{MatMul}}$ (linear kernels) and from batched lookups (nonlinear maps). We first perform a *evaluation reduction* to bind all claims to a common point (or a few points) and compress the values using the Fiat–Shamir coefficients: $v_{\text{agg}} = \sum_j \lambda_j v_j$.

Because our PCS is non-homomorphic, we do not combine commitments algebraically. Instead, using the *same* coefficients $\{\lambda_j\}$ we build a fresh aggregated code-word $\mathbf{M}^* = \sum_j \lambda_j \mathbf{M}_j$, commit to it (yielding σ^*), and at $\Theta(\lambda)$ sampled coordinates open Merkle-tree paths for $\mathbf{M}^*$ and for each $\mathbf{M}_j$ to certify the linear relation $\mathbf{M}^*[r, c] = \sum_j \lambda_j \mathbf{M}_j[r, c]$. These checks reuse the same sumcheck-friendly constraints and link v_{agg} to σ^* without relying on homomorphism. Both linear and non-linear families flow through this pipeline, so the accumulator carries only $(\sigma^*, v_{\text{agg}})$ and a transcript digest; in the end, we discharge them with a constant number of batched

openings. The prover’s work is linear in the total encoded size, while verifier time and proof size are logarithmic in the number of aggregated items [8].

Sumcheck-friendly hashing and challenges. All challenges within the step (for sumcheck and aggregation) are derived via the Fiat–Shamir transformation from a transcript that binds $(i+1, z_0, z_i, h_i)$ and the relevant commitments. Following [8], we instantiate a sumcheck-friendly permutation (e.g., MiMC), so that the in-circuit verification of hash rounds decomposes into simple linear maps.

Tree-based organization. To reduce recursion depth, we can organize steps in a k -ary tree: leaves are per-iteration step proofs; each internal node verifies its k children in-circuit and reapplies the reduction/aggregation pipeline to emit one parent proof. The depth is $\lceil \log_k T \rceil$, and the root proof, together with the same constant number of batched openings, is the final object of verification [29].

5.1. Putting Everything Together

At the parent invocation of the augment function F_A over k child steps, we use the following *inputs*: (i) the k child transcripts and public I/O $\{(\pi_j, \text{io}_j, h_j)\}_{j=1}^k$, where each io_j contains the pre-aggregation commitments for that step (parameter and lookup commitments), the *committed* next-iteration parameters (e.g., $\text{cm}_\theta^{(i_j+1)}$), and that step’s aggregated triple $(\sigma_j^*, r_{\text{agg},j}, v_{\text{agg},j})$; (ii) Fiat–Shamir challenge $\{\lambda_j\}$, the sampled coordinates $\mathcal{S}$ of size $\Theta(\lambda)$, and the Merkle authentication paths for the constituent codewords at $\mathcal{S}$; (iii) hash advice for the k PBPTT transcripts and for the current aggregation (to be checked by sumcheck).

Inside F_A , the value side is first aligned by an evaluation reduction that binds all per-kernel claims to a common point r_{agg} (or a small set of points) and compresses them by a random linear combination, $v_{\text{agg}} = \sum_{j=1}^k \lambda_j v_j$. Because the underlying PCS is non-homomorphic, the commitment side is handled by constructing a fresh aggregated codeword with the *same* coefficients,

$$\mathbf{M}^* = \sum_{j=1}^k \lambda_j \mathbf{M}_j, \quad \sigma^* \leftarrow \text{PCS.Commit}(\mathbf{M}^*),$$

and certifying, *in circuit and without calling* PCS.Open , that for every $(r, c) \in \mathcal{S}$ the relation $\mathbf{M}^*[r, c] = \sum_{j=1}^k \lambda_j \mathbf{M}_j[r, c]$, holds via the provided Merkle paths. This links the reduced value v_{agg} to the new commitment σ^* while deferring openings.

The step verifier then rechecks all k child proofs in circuit and *enforces* acceptance before proceeding. For a node, the step additionally enforces the PBPTT constraints (forward, backward, and the parameter update) that relate the input commitment $\text{cm}_\theta^{(i)}$ to the *committed* output $\text{cm}_\theta^{(i+1)}$, producing the per-kernel evaluation claims consumed by the reduction above. For an internal node, no new PBPTT computation is introduced—only child-proof verification and aggregation are performed.

Next, the hash permutations that feed Fiat–Shamir are verified by sumcheck: one family covers the k PBPTT transcripts, and another covers the current aggregation, using a sumcheck-friendly algebraic hash (e.g., MiMC) so

that the permutation rounds arithmetize to simple linear maps.

Finally, F_A records the aggregated triple $(\sigma^*, r_{\text{agg}}, v_{\text{agg}})$, updates a domain-separated transcript digest, and produces the next accumulator with a fixed tiny schema (counter, digest, aggregated pair, minimal metadata). For leaves, the output parameter commitment $\text{cm}_\theta^{(i+1)}$ simultaneously becomes the next step’s input. The outputs of this invocation are the combined verifier result (enforced to be 1 in circuit), the sumcheck proofs for the k PBPTT hashes and for the current aggregation hash, the aggregated triple $(\sigma^*, r_{\text{agg}}, v_{\text{agg}})$, and the updated accumulator. Crucially, no per-iteration PCS openings are performed inside the recursion; because each step publishes only the aggregated triple and the list of constituent commitments as public I/O, all evaluation openings are deferred to the end and discharged once with a constant number of batched openings.

6. SUMMER: A Recursive Framework of RNN Training

In this section, we present SUMMER, the first end-to-end verifiable training system for RNNs. We first propose a model-agnostic definition for time series models that covers our RNN instantiation and can be extended to transformer-based large language models (LLM), and then we describe the concrete design and implementation of SUMMER.

6.1. A Generic Definition

We adopt the zero-knowledge proof-of-training interface of Abbaszadeh et al. [8] and generalize it to our framework. Let the following step function $z_{i+1} = F(z_i, \omega_i)$ describe training, where the public state z_i includes model parameters θ_i and ω_i is the private per-step witness derived from the dataset and randomness.

Definition 2. *Given a time-series model, a generic state function $\mathcal{F}$ is defined as a deterministic mapping that updates the model parameters $\theta_{i+1} = \mathcal{F}(\theta_i, \omega_i)$, a zero-knowledge proof of training is a tuple of PPT algorithms (KeyGen, DataCom, ParCom, Prove, Verify. Among them, $\text{pp} \leftarrow \text{KeyGen}(1^\lambda)$ computes the public parameters, $\sigma_{\mathcal{D}} \leftarrow \text{DataCom}(\mathcal{D}, r_{\mathcal{D}})$ commits to the (entire) training dataset $\mathcal{D}$, $\sigma_\theta \leftarrow \text{ParCom}(\theta; r_\theta)$: commit to the updated model parameters. $(\pi_i, \sigma_{\theta_i}) \leftarrow \text{Prove}(\text{pp}, i, \sigma_{\mathcal{D}}, \sigma_{\theta_0}, \sigma_{\theta_{i-1}}, \theta_{i-1}, \pi_{i-1})$ computes proof a single training step is correct. The prover outputs a new commitment σ_{θ_i} and a proof π_i , and $\{0, 1\} \leftarrow \text{Verify}(i, \sigma_{\mathcal{D}}, \sigma_{\theta_0}, \sigma_{\theta_i}, \pi_i)$: accepts if and only if the above step relation holds with respect to the commitments.*

This definition satisfies the essential properties of completeness, knowledge soundness and zero-knowledge as defined in [8] Section E.1.

6.1.1. Iteration-succinct end-to-end proof. The above definition is for single-step training. An end-to-end zkPoT for the T steps provides a composition mechanism via IVC that yields a final *single* proof Π_T for $(\sigma_{\mathcal{D}}, \sigma_{\theta_0}, \sigma_{\theta_T})$ with a verification cost and a proof size independent of

T. We use this generalized form in SUMMER with time-indexed witnesses and commitments.

6.2. Implementation of SUMMER

SUMMER implements an end-to-end verifiable training pipeline that turns each PBPTT into a succinct step proof and then composes all steps recursively. This implementation follows the technical components proposed in Section B and the IVC interface in Section 5, and instantiates the design of the time series model with RNNs as the running case.

6.2.1. Data/model commitments and Fiat-Shamir transformation. At initialization, we commit to the training inputs and labels, the initial model parameters, and the precomputed lookup tables for nonlinearities through a PCS. Each PBPTT iteration then packs the iteration’s witness into a single vector that includes the lookup indices/values used by the activations and the updated parameters; this vector is also committed. All prover challenges are derived from a single Fiat-Shamir transcript driven by a sumcheck-friendly hash so that every random evaluation point, Merkle query coordinate, and aggregation coefficient is bound to the public transcript and ensures consistency across the circuit.

6.2.2. Per-iteration proving. After running PBPTT (Protocol B), the prover generates the step proof. ALL linear relations are reduced to matrix multiplication and are proved by the $\text{Sum}_{\text{MatMul}}$ subprotocol. Nonlinear constraints from activation functions are enforced with TaSSLE-style lookups *against the committed lookup tables*: the prover provides indexed queries and proves them with a logarithmic-derivative lookup that composes with sumcheck proofs, while deferring commitment to the recursive aggregation stage. Throughout the step we collect only a small set of random evaluation claims (σ, r, v) for matrices/vectors, the verifier will verify the corresponding openings once at the end of IVC.

6.2.3. Recursive Composition. Every k step, SUMMER invokes the augment function F_A to fold the child steps into one parent. Values are combined by reducing the evaluation to a common point and a random linear combination; commitments are combined by the multivariate commitment aggregation method [8]. The parent also invokes the k child verification in-circuit and verifies the Fiat-Shamir hash permutations through small sumcheck instances over a MiMC-style [30] hash function. The parent outputs a tiny accumulator that carries a counter, a transcript digest, and one aggregated $(\sigma^*, r_{\text{agg}}, v_{\text{agg}})$. No openings occur within the recursion; all openings are batched once at the root.

6.2.4. Optimized Engineering. In the IVC process, lookup table commitments are cached and their Merkle roots reused across iterations. Commitment encoding and sumcheck instances use parallel loops, while intermediate tensors are released per-iteration to keep memory low. The prover maintains a compact transcript of step proofs and aggregation proofs and records only the minimal advice required by the parent F_A .

6.2.5. Outputs. The output of SUMMER consists of a final commitment to the trained parameters, one recursive proof that certifies to all iterations, and a constant number of batched opening proofs that validate all deferred evaluation claims. Verifier time and proof size are independent of the number of iterations, while the prover time at each step is dominated by the parameters commitment.

Combining all the ingredients discussed above, we have the full description of SUMMER as in Protocol 6.2.5

Theorem 2. SUMMER (Protocol 6.2.5) is a zero-knowledge proof of training for time-series model and satisfies completeness, knowledge soundness, and zero-knowledge.

Completeness follows directly since each PBPTT.Prove call produces valid sumcheck transcripts for the linear and nonlinear relations, and the aggregation step F_A only folds already verified children and Merkle-certified codewords; by induction the final root proof certifies the correctness of the entire training. For knowledge soundness, if an adversary outputs an accepting proof, then by the extractability of PCS and the knowledge soundness of $\text{Sum}_{\text{MatMul}}$, LogUp, and GKR, one can extract the committed parameters and lookup witnesses for each iteration. The recursive aggregation preserves extractability, so running the extractors iteratively recovers the full training trace, ensuring that no accepting proof can exist without knowledge of a valid BPTT execution. Zero-knowledge is immediate from the hiding property of PCS and Merkle commitments, and from the zero-knowledge simulators of $\text{Sum}_{\text{MatMul}}$, LogUp, and GKR; the aggregation adds only randomized linear combinations, so the entire proof transcript leaks nothing beyond the public commitments. The security proof can be found in Appendix C.

7. Implementation and Evaluation

7.1. Implementation

7.1.1. Framework Setup. We implemented SUMMER mainly in C++. The sumcheck and GKR protocols are built on open-source libraries [3]. We use Orion [10] as our multivariate polynomial commitment scheme due to its linear prover time and polylogarithmic verifier time and proof size. We adopt TaSSLE [27] as our look-up argument, which provides minimal commitment costs. All protocols work on arbitrary finite fields; in our implementation, we instantiate them in the quadratic extension field $\mathbb{F}_{p^2}$ with $p = 2^{61} - 1$ a Mersenne prime. For cryptographic primitives, we use MiMC [30] with 80 rounds and SHA-256 as our hash functions, which together provide a security level of $\lambda = 100$ bits.

All experiments were run on a Linux server with an AMD EPYC 9374F 3.85GHz 32-Core Processor and 1.5TB RAM. We set the quantization parameter to $Q = 32$, which quantized each real number as a signed 32-bit fixed-point integer. After symmetric clipping, the quantized values lie in $[-2^{31}, 2^{31} - 1]$ and are then embedded in $\mathbb{F}_p$. For memory and fair-comparison considerations, we configure the recursion tree with arity $k = 10$ and depth $d = 2$, yield $10^2 = 100$ training iterations. This setting is configurable: It extends naturally to larger recursion

Protocol 6.2.5. SUMMER Zero-knowledge Proof of Training for RNN

Parameters. Let $\text{PCS} = (\text{KeyGen}, \text{Commit}, \text{Open}, \text{Verify})$ be a (non-homomorphic) multivariate polynomial commitment scheme; $\text{Sum}_{\text{MatMul}}$ denote the time-optimal sumcheck subprotocol for matrix multiplication; LogUp denote the logarithmic-derivative lookup argument; GKR be a generic sumcheck-based zk system; $\text{PBPTT} = (\text{Prove}, \text{Verify})$ be the per-iteration step protocol (Protocol B, using $\text{Sum}_{\text{MatMul}} + \text{LogUp}$); FS be a Fiat-Shamir transcript over a sumcheck-friendly hash. Fix recursion arity $k \geq 2$.

Preprocessing. (i) $\text{pp} \leftarrow \text{PCS.KeyGen}(1^\lambda)$. (ii) Dataset commitment: $\sigma_{\mathcal{D}} \leftarrow \text{DataCom}(\mathcal{D}; r_{\mathcal{D}})$. (iii) Initial parameters: $\sigma_{\theta_0} \leftarrow \text{ParCom}(\theta_0; r_{\theta_0})$. (iv) One-time commitments for activation lookup tables used by LogUp : $\sigma_{T,\text{act}}, \sigma_{T,\text{Softmax}} \leftarrow \text{ParCom}(T_{\text{act}} \parallel T_{\text{Softmax}}; r_T)$. (v) Initialize FS with public commitments/metadata; set base proof Π_0 and accumulator $\text{acc}_0 = (\text{ctr} = 0, \text{digest}, \perp)$.

$(\sigma_{\theta_i}, \Pi_i) \leftarrow \text{Prove}(i, \text{pp}, \sigma_{\mathcal{D}}, \sigma_{\theta_0}, \sigma_{\theta_{i-1}}, \theta_{i-1}, \Pi_{i-1})$.

- 1) Run one BPTT iteration on $(\theta_{i-1}, \mathcal{D})$ to obtain θ_i and the packed step witness wit_i . Commit:

$$\sigma_{\theta_i} \leftarrow \text{ParCom}(\theta_i; r_{\theta_i}), \quad \sigma_{\text{wit},i} \leftarrow \text{ParCom}(\text{wit}_i; r_{\text{wit},i}).$$

- 2) Invoke PBPTT.Prove using $\text{Sum}_{\text{MatMul}}$ for all linear maps and LogUp for act/Softmax lookups *against* $(\sigma_{T,\text{act}}, \sigma_{T,\text{Softmax}})$. This yields a sumcheck bundle that exposes only random evaluation claims $\{(\sigma_j, r_j, v_j)\}_j$ for the matrices/vectors that appeared during the checks.
- 3) When k fresh children are buffered, run the augment function F_A : derive FS challenges, batch-verify the k child step transcripts *in-circuit* via PBPTT.Verify ; align value claims to a common point r_{agg} and compress $v_{\text{agg}} = \sum_{j=1}^k \lambda_j v_j$; build an aggregated codeword $M^* = \sum_{j=1}^k \lambda_j M_j$ and set $\sigma^* \leftarrow \text{PCS.Commit}(M^*)$; certify on a public sample set $\mathcal{S}$ that $M^*[r, c] = \sum_j \lambda_j M_j[r, c]$ via Merkle paths; verify the hash-permutation rounds that feed FS by small sumchecks; update the accumulator to $\text{acc}_i = (\text{ctr}+1, \text{digest}', (\sigma^*, r_{\text{agg}}, v_{\text{agg}}))$.
- 4) Assemble

$$\Pi_i = (\Pi_i^{\text{sc}}, \Pi_i^{\text{agg}}, \Pi_i^{\text{open}}, \text{acc}_i),$$

where Π_i^{sc} is produced by PBPTT.Prove (linear layers via $\text{Sum}_{\text{MatMul}}$, nonlinearities via LogUp against $\sigma_{T,\text{act}}, \sigma_{T,\text{Softmax}}$, plus the sumchecks for the FS permutations); Π_i^{agg} is produced by the parent fold F_A over k children (evaluation reduction to r_{agg} , new σ^* , Merkle checks on $\mathcal{S}$, and hash-permutation sumchecks); Π_i^{open} is empty for internal nodes and becomes the final batched openings at the root.

$\{0, 1\} \leftarrow \text{Verify}(i, \text{pp}, \sigma_{\mathcal{D}}, \sigma_{\theta_0}, \sigma_{\theta_i}, \Pi_i)$.

- 1) Parse $\Pi_i = (\Pi_i^{\text{sc}}, \Pi_i^{\text{agg}}, \Pi_i^{\text{open}}, \text{acc}_i)$ and recompute the Fiat-Shamir and Merkle hashes from the public inputs and acc_i .
- 2) Run PBPTT.Verify (and any auxiliary GKR.Verify) on Π_i^{sc} with respect to $(\sigma_{\mathcal{D}}, \sigma_{\theta_0}, \sigma_{\theta_i})$ to obtain the transcript-bound commitment-evaluation claims $\{(\sigma_j, x_j, y_j)\}_j$.
- 3) Verify Π_i^{agg} against $\{(\sigma_j, x_j, y_j)\}_j$ to check the current fold and recover the next aggregated instance encoded in acc_i . If i is the recursion root, additionally verify the batched evaluation openings in Π_i^{open} via PCS.Verify . Accept iff all checks pass.

parameters, e.g., $k = 20$ and $d = 4$, which correspond to $20^4 = 160,000$ iterations.

7.1.2. Scalable Design. Our framework is designed to be scalable and general and supports all the computations required by existing RNN models. In particular, SUMMER handles standard matrix-matrix, matrix-vector multiplications, as well as element-wise operations for arbitrary model dimensions. Beyond linear calculations, SUMMER also implements non-linear activation functions such as ReLU, tanh and Softmax. For each of these operators, we provide both forward evaluation and backward gradient computation, together with the corresponding proof-generation interfaces, enabling users to flexibly compose and verify complete model executions, thereby achieving true end-to-end model verification.

7.1.3. Model Choice. For evaluation, we select the Minimal Character Level Language Model with a Vanilla Recurrent Neural Network (Min-Char-RNN) [11] as our

benchmark. This is a classic RNN model consisting of an input layer, a single hidden layer, and an output layer, with activation functions tanh and Softmax. The model is highly scalable: all the dimensions of the layer and the training time steps can be customized. In our experiments, we trained Min-Char-RNN under multiple parameter settings of increasing dimension and time steps (cd. Table 1, Table 2), which allow us to evaluate both the scalability and efficiency of our framework.

7.2. Performance of SUMMER

7.2.1. Performance of PBPTT. We first benchmark the prover time of PBPTT, which represents the cost of generating a complete step proof for a single iteration. We report the average prover time for 100 training iterations. As shown in Table 1, the proof is divided into three stages: *Update*, *Backward*, and *Forward*. We evaluated models ranging from small (50k parameters) to large (12M parameters). The results show that most of the

PBPTT prover time comes from the *Forward* stage, mainly because it includes the proof of activation functions via lookup arguments. Notably, even for the largest model with 12M parameters, generating a single iteration proof takes only 11.136 seconds. This shows that SUMMER scales well with model dimension and remains practical even for multi-million parameter RNNs.

7.2.2. Overall Performance. Beyond the per-step analysis, we also evaluate the end-to-end proving pipeline of SUMMER, as reported in Table 2. At the beginning of each iteration, the prover calls the PBPTT prover, which involves generating parameter commitments, witness commitments, and LogUpcommitments. These initialization costs are averaged and reported as *Commitment* time. The PBPTT prover then executes the RNN training computation and produces a step proof, which is reflected in *PBPTT* time. The commitment and proof transcript generated for each step are treated as leaves in the recursion tree. In every k steps, the prover invokes the recursion tree’s augment function to aggregate child proofs. This process includes forming a linear combination of evaluation claims and issuing queries, measured as the *Aggregate* cost. The *Prove aggr.* stage corresponds to proving this aggregation, while the *IVC* stage accounts for intermediate hash-based sumchecks and the in-circuit verification of child proofs. All of these aggregation-related costs are amortized over the total number of iterations. Finally, the entire pipeline outputs the total prover time, along with constant-size proof transcripts whose verification time and proof size remain independent of the number of iterations.

From a systems perspective, the overall proving pipeline exhibits remarkable stability. The *Total prover time* remains small even for very large models: for instance, the 12M-parameter RNN completes a full iteration proof in only 75.8 seconds. The intermediate recursion stage, implemented through the augment function (i.e. *Aggregate*, *Prove aggr.*, and *IVC*), contributes only a minor and stable overhead across all model sizes, typically less than 10% of the total prover time. The *Commitment* phase consistently dominates the prover cost. This overhead is inherent and unavoidable, as every iteration must commit to fresh parameters and witness data. Importantly, the verifier time (≈ 20 ms) and proof size (≈ 164 KB) remain constant, independent of the model size and the number of iterations. Finally, memory usage grows with parameter size but under control: even at the largest scale, peak memory reaches about 324GB, which is well within the capacity of modern servers relative to the massive underlying RNN computations. These results confirm that SUMMER achieves scalable proving performance while ensuring succinct and stable verification.

7.2.3. Proving Overhead. Figure 1 plots the ratio of the total proving time per-iteration to the training time per-iteration across all model sizes. The overhead starts high on small models about $81\times$ for the 50k-parameter model at $T = 64$ and $23\times$ at $T = 256$.² As the parameter grows, the factor shrinks rapidly: $4.7\times/1.7\times$ at 1.5M parameters and just $2.2\times/0.6\times$ at 12M parameters ($T=64/256$, respectively). In other words, once the network reaches practical scale, proof generation adds only

2. Ratios are computed row-wise from Table 2.

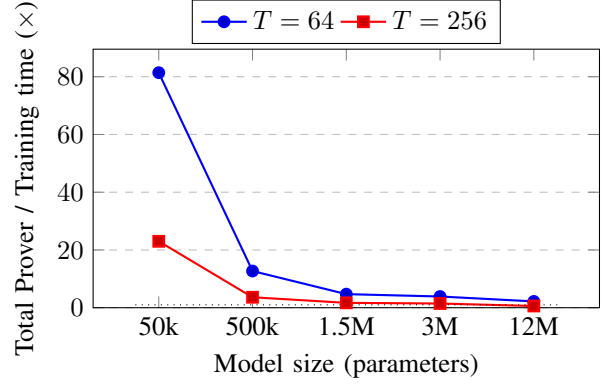


Figure 1: Relative overhead of proof generation: ratio of *Total Prover* to *Training* time across model sizes (lower is better).

a small constant multiple, even fall *below* the cost of training itself.

7.3. Comparison with Baseline

We compare SUMMER with Nova [12], where computations must be compiled into RICS, causing constraints—and thus proving time, setup, folding, verifier cost, and memory—to grow linearly with model size and sequence length. By contrast, SUMMER shows only mild, stable growth in prover time as sequences extend, with per-iteration costs varying only slightly across time steps.

Metric	Time Steps (T)					
	2 ⁴	2 ⁵	2 ⁶	2 ⁷	2 ⁸	2 ⁹
Total Prover (s)						
Nova	11.57	18.01	32.31	60.90	107.71	226.03
SUMMER	18.76	19.09	19.71	20.39	22.20	26.52
Total Verifier (s)						
Nova	2.19	2.17	4.96	7.47	20.13	34.20
SUMMER	0.02	0.02	0.02	0.02	0.02	0.02
Proof size (KB)						
Nova	14.82	14.83	15.19	15.53	15.53	16.24
SUMMER	164.59	164.59	164.59	164.59	164.59	164.59

TABLE 3: Comparison of Nova and SUMMER (dimension $256 \times 512 \times 256$, model size 525,056).

Table 3 summarizes the detailed comparison between Nova and SUMMER in the same 500k parameter setting. Nova requires a one-time preprocessing step to sample its public parameters, which can be expensive in practice. Beyond this preprocessing, Nova’s per-iteration proving time, folding time, verifier time, and peak memory all scale linearly in T . In contrast, SUMMER maintains stable verifier cost and succinct proofs, with only mild growth in prover time. At $T = 512$, SUMMER achieves an increase in $8.5\times$ total prover time and reduces memory usage by $11.6\times$ compared to Nova.

Figures 2 and 3 visualize the comparison: while Nova’s prover time and memory usage grow sharply with T , SUMMER remains efficient and memory-stable across all evaluated settings.

Metric	50k		500k		1.5M		3M		12M	
	T=64	T=256	T=64	T=256	T=64	T=256	T=64	T=256	T=64	T=256
Update	0.005	0.005	0.051	0.050	0.154	0.154	0.298	0.298	1.399	1.402
Backward	0.008	0.024	0.046	0.106	0.129	0.253	0.201	0.335	0.841	1.120
Forward	1.679	2.835	2.717	4.540	3.282	4.575	3.908	5.353	5.574	8.614
Total Prover	1.692	2.864	2.814	4.696	3.566	4.982	4.407	5.987	7.815	11.136

TABLE 1: PBPTT Prover time for Min-Char-RNN (seconds).

Metric	50k		500k		1.5M		3M		12M	
	T=64	T=256	T=64	T=256	T=64	T=256	T=64	T=256	T=64	T=256
Training	0.193	0.771	1.555	6.121	4.995	18.750	7.781	30.899	31.521	128.175
Commitment	5.82	6.48	8.44	9.04	11.31	17.69	16.96	29.55	50.82	54.56
PBPTT	1.69	2.86	2.81	4.70	3.57	4.98	4.41	5.99	7.82	11.14
Aggregate	0.13	0.12	0.16	0.16	0.22	0.33	0.33	0.53	0.99	1.01
Prove aggr.	7.81	8.00	8.01	7.99	8.03	8.07	8.05	8.08	7.98	7.96
IVC	0.25	0.24	0.28	0.29	0.34	0.45	0.45	0.66	1.12	1.14
Total Prover	15.71	17.70	19.71	22.17	23.47	31.52	30.19	44.81	68.72	75.80
Total Verifier	0.02	0.02	0.02	0.02	0.02	0.02	0.02	0.02	0.02	0.03
Proof size (KB)	164.59	164.59	164.59	164.59	164.59	164.59	164.59	164.59	164.59	164.59
Peak Mem. (GB)	81.76	84.65	94.51	98.06	108.98	142.19	137.53	201.61	304.86	323.68

TABLE 2: End-to-end proving statistics for Min-Char-RNN (seconds unless noted).

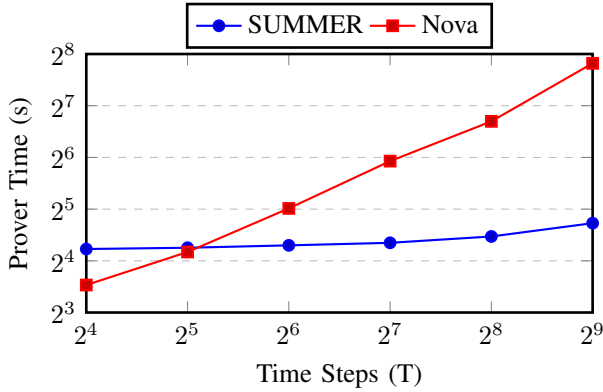


Figure 2: Prover time vs. time steps.

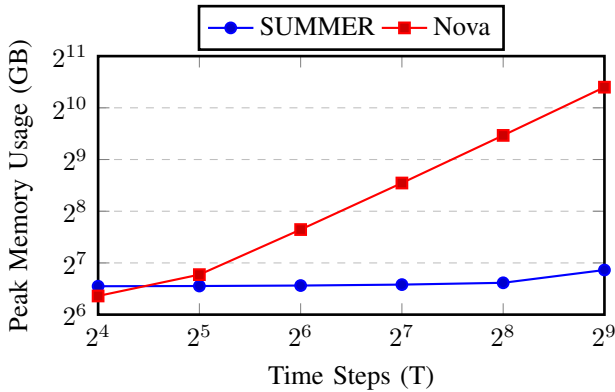


Figure 3: Peak memory usage vs. time steps.

References

- [1] H. Sun, T. Bai, J. Li, and H. Zhang, “Zkd1: Efficient zero-knowledge proofs of deep learning training,” *IEEE Transactions on Information Forensics and Security*, 2024.
- [2] S. Garg, A. Goel, S. Jha, S. Mahlouljifar, M. Mahmoody, G.-V. Policharla, and M. Wang, “Experimenting with zero-knowledge proofs of training,” in *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, 2023, pp. 1880–1894.
- [3] T. Liu, X. Xie, and Y. Zhang, “zkcn: Zero knowledge proofs for convolutional neural network predictions and accuracy,” in *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, 2021, pp. 2968–2985.
- [4] C. Weng, K. Yang, X. Xie, J. Katz, and X. Wang, “Mystique: Efficient conversions for {Zero-Knowledge} proofs with applications to machine learning,” in *30th USENIX Security Symposium (USENIX Security 21)*, 2021, pp. 501–518.
- [5] B.-J. Chen, S. Waiwitlikhit, I. Stoica, and D. Kang, “Zkml: An optimizing system for ml inference in zero-knowledge proofs,” in *Proceedings of the Nineteenth European Conference on Computer Systems*, 2024, pp. 560–574.
- [6] H. Sun, J. Li, and H. Zhang, “zkllm: Zero knowledge proofs for large language models,” in *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, 2024, pp. 4405–4419.
- [7] W. Qu, Y. Sun, X. Liu, T. Lu, Y. Guo, K. Chen, and J. Zhang, “zkpgt: An efficient non-interactive zero-knowledge proof framework for llm inference,” in *34th USENIX Security Symposium (USENIX Security 25)*, 2025.
- [8] K. Abbaszadeh, C. Pappas, J. Katz, and D. Papadopoulos, “Zero-knowledge proofs of training for deep neural networks,” in *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, 2024, pp. 4316–4330.
- [9] U. Haböck, “Multivariate lookups based on logarithmic derivatives,” *Cryptology ePrint Archive*, 2022.

- [10] T. Xie, Y. Zhang, and D. Song, “Orion: Zero knowledge proof with linear prover time,” in *Annual International Cryptology Conference*. Springer, 2022, pp. 299–328.
- [11] A. Karpathy, “The unreasonable effectiveness of recurrent neural networks,” <https://karpathy.github.io/2015/05/21/rnn/>, 2015.
- [12] A. Kothapalli, S. Setty, and I. Tzialla, “Nova: Recursive zero-knowledge arguments from folding schemes,” in *Annual International Cryptology Conference*. Springer, 2022, pp. 359–388.
- [13] J. L. Elman, “Finding structure in time,” *Cognitive science*, vol. 14, no. 2, pp. 179–211, 1990.
- [14] P. J. Werbos, “Backpropagation through time: what it does and how to do it,” *Proceedings of the IEEE*, vol. 78, no. 10, pp. 1550–1560, 2002.
- [15] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” *arXiv preprint arXiv:1412.6980*, 2014.
- [16] J. Duchi, E. Hazan, and Y. Singer, “Adaptive subgradient methods for online learning and stochastic optimization,” *Journal of machine learning research*, vol. 12, no. 7, 2011.
- [17] G. Hinton, “Neural networks for machine learning, lecture 6e: Rmsprop,” Coursera video lectures, 2012, uRL: http://www.cs.toronto.edu/~tijmen/csc321/slides/lecture_slides_lec6.pdf.
- [18] R. Pascanu, T. Mikolov, and Y. Bengio, “On the difficulty of training recurrent neural networks,” in *Proceedings of the 30th International Conference on Machine Learning (ICML)*, 2013, pp. 1310–1318.
- [19] A. Fiat and A. Shamir, “How to prove yourself: Practical solutions to identification and signature problems,” in *Conference on the theory and application of cryptographic techniques*. Springer, 1986, pp. 186–194.
- [20] P. Valiant, “Incrementally verifiable computation or proofs of knowledge imply time/space efficiency,” in *Theory of Cryptography: Fifth Theory of Cryptography Conference, TCC 2008, New York, USA, March 19-21, 2008. Proceedings 5*. Springer, 2008, pp. 1–18.
- [21] R. C. Merkle, “A digital signature based on a conventional encryption function,” in *Conference on the theory and application of cryptographic techniques*. Springer, 1987, pp. 369–378.
- [22] C. Lund, L. Fortnow, H. Karloff, and N. Nisan, “Algebraic methods for interactive proof systems,” *Journal of the ACM (JACM)*, vol. 39, no. 4, pp. 859–868, 1992.
- [23] S. Goldwasser, Y. T. Kalai, and G. N. Rothblum, “Delegating computation: interactive proofs for muggles,” *Journal of the ACM (JACM)*, vol. 62, no. 4, pp. 1–64, 2015.
- [24] S. Setty, J. Thaler, and R. Wahby, “Unlocking the lookup singularity with lasso,” in *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Springer, 2024, pp. 180–209.
- [25] S. Setty and J. Thaler, “Twist and shout: Faster memory checking arguments via one-hot addressing and increments,” *Cryptology ePrint Archive*, 2025.
- [26] S. Papini and U. Haböck, “Improving logarithmic derivative lookups using gkr,” *Cryptology ePrint Archive*, 2023.
- [27] D. Dore, “Tassle: Lasso for the commitment-phobic,” *Cryptology ePrint Archive*, 2024.
- [28] J. Thaler, “Time-optimal interactive proofs for circuit evaluation,” in *Annual Cryptology Conference*. Springer, 2013, pp. 71–89.
- [29] N. Bitansky, R. Canetti, A. Chiesa, and E. Tromer, “Recursive composition and bootstrapping for snarks and proof-carrying data,” in *Proceedings of the forty-fifth annual ACM symposium on Theory of computing*, 2013, pp. 111–120.
- [30] M. Albrecht, L. Grassi, C. Rechberger, A. Roy, and T. Tiessen, “Mimc: Efficient encryption and cryptographic hashing with minimal multiplicative complexity,” in *International Conference on the Theory and Application of Cryptology and Information Security*. Springer, 2016, pp. 191–219.
- [31] T. Xie, J. Zhang, Y. Zhang, C. Papamanthou, and D. Song, “Libra: Succinct zero-knowledge proofs with optimal prover computation,” in *Advances in Cryptology—CRYPTO 2019: 39th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18–22, 2019, Proceedings, Part III 39*. Springer, 2019, pp. 733–764.

Appendix A.

Additional Preliminaries

In this section we introduce the foundational background used throughout the paper. We first present the full Backpropagation Through Time (BPTT) algorithm. We then give formal definitions of interactive proofs/arguments, polynomial commitment schemes (PCS), and incrementally verifiable computation (IVC). Finally, we provide detailed protocols for the sumcheck and GKR frameworks.

A.1. RNNs and BPTT

The Backpropagation Through Time (BPTT) algorithm is shown in Algorithm 1. Here Act is an element-wise nonlinearity, “ $\circ$ ” denotes the Hadamard product, and we average gradients over the sequence length before the SGD update. With softmax and cross-entropy, the output-layer residual simplifies to $\delta_i^{(t)} = \hat{\mathbf{y}}_i^{(t)} - \mathbf{y}_i^{(t)}$.

Notation and dimensions. We use n for the input size, h for the hidden size, k for the output size, and T for the sequence length (time steps). Vectors are column vectors; biases broadcast along the feature dimension. At training iteration i and time step t ,

$$\begin{aligned} \mathbf{x}_i^{(t)} &\in \mathbb{R}^n, & \mathbf{h}_i^{(t)} &\in \mathbb{R}^h \text{ } (\mathbf{h}_i^{(0)} \text{ given}), \\ \mathbf{a}_i^{(t)} &\in \mathbb{R}^h, & \mathbf{z}_i^{(t)} &\in \mathbb{R}^k, & \hat{\mathbf{y}}_i^{(t)} &\in \mathbb{R}^k. \end{aligned}$$

Parameters and their shapes are

$$\begin{aligned} \mathbf{W}_x &\in \mathbb{R}^{h \times n}, & \mathbf{W}_h &\in \mathbb{R}^{h \times h}, \\ \mathbf{W}_y &\in \mathbb{R}^{k \times h}, & \mathbf{b}_1 &\in \mathbb{R}^h, & \mathbf{b}_2 &\in \mathbb{R}^k. \end{aligned}$$

In the backward pass we use

$$\begin{aligned} \delta_i^{(t)} &= \hat{\mathbf{y}}_i^{(t)} - \mathbf{y}_i^{(t)} \in \mathbb{R}^k, \\ \mathbf{h}_i^{\prime(t)} &\in \mathbb{R}^h, & \mathbf{a}_i^{\prime(t)} &\in \mathbb{R}^h. \end{aligned}$$

Gradients satisfy

$$\begin{aligned} \nabla \mathbf{w}_y &\in \mathbb{R}^{k \times h}, & \nabla \mathbf{w}_h &\in \mathbb{R}^{h \times h}, & \nabla \mathbf{w}_x &\in \mathbb{R}^{h \times n}, \\ \nabla \mathbf{b}_1 &\in \mathbb{R}^h, & \nabla \mathbf{b}_2 &\in \mathbb{R}^k, \end{aligned}$$

and we use a scalar learning rate η with averaging over T before the update.

Algorithm 1 Backpropagation Through Time (BPTT)

Input: Weights $\mathbf{W}_{y,i-1}, \mathbf{W}_{h,i-1}, \mathbf{W}_{e,i-1}$; biases $b_{1,i-1}, b_{2,i-1}$; initial hidden state $\mathbf{h}_i^{(0)}$; inputs $\{\mathbf{x}_i^{(t)}\}_{t=1}^T$ and labels $\{\mathbf{y}_i^{(t)}\}_{t=1}^T$

Output: Updated weights $\mathbf{W}_{y,i}, \mathbf{W}_{h,i}, \mathbf{W}_{e,i}$ and $b_{1,i}, b_{2,i}$

INITIALIZE

1: $\nabla \mathbf{W}_{y,i} \leftarrow 0, \nabla \mathbf{W}_{h,i} \leftarrow 0, \nabla \mathbf{W}_{e,i} \leftarrow 0, \nabla b_{1,i} \leftarrow 0, \nabla b_{2,i} \leftarrow 0$
2: $\mathcal{L} \leftarrow 0$

FORWARD PASS

3: **for** $t = 1$ to T **do**
4: $\mathbf{a}_i^{(t)} \leftarrow \mathbf{W}_{h,i-1} \mathbf{h}_i^{(t-1)} + \mathbf{W}_{e,i-1} \mathbf{x}_i^{(t)} + b_{1,i-1} \triangleright \mathbf{a}_i^{(t)} \in \mathbb{R}^h$
5: $\mathbf{h}_i^{(t)} \leftarrow \text{Act}(\mathbf{a}_i^{(t)})$
6: $z_i^{(t)} \leftarrow \mathbf{W}_{y,i-1} \mathbf{h}_i^{(t)} + b_{2,i-1}$
7: $\hat{\mathbf{y}}_i^{(t)} \leftarrow \text{softmax}(z_i^{(t)})$
8: $\mathcal{L}^{(t)} \leftarrow - \sum_k y_{i,k}^{(t)} \log(\hat{y}_{i,k}^{(t)})$
9: $\mathcal{L} \leftarrow \mathcal{L} + \mathcal{L}^{(t)}$
10: **end for**

BACKWARD PASS

11: **for** $t = T$ **downto** 1 **do**
12: $\delta_i^{(t)} \leftarrow \hat{\mathbf{y}}_i^{(t)} - \mathbf{y}_i^{(t)} \triangleright \delta_i^{(t)} \in \mathbb{R}^K$
13: $\nabla \mathbf{W}_{y,i} \leftarrow \delta_i^{(t)} (\mathbf{h}_i^{(t)})^\top$
14: $\nabla b_{2,i} \leftarrow \delta_i^{(t)}$
15: **if** $t = T$ **then**
16: $\mathbf{h}_i'^{(t)} \leftarrow \mathbf{W}_{y,i-1}^\top \delta_i^{(t)}$
17: **else**
18: $\mathbf{h}_i'^{(t)} \leftarrow \mathbf{W}_{y,i-1}^\top \delta_i^{(t)} + \mathbf{W}_{h,i-1}^\top \mathbf{a}_i'^{(t+1)}$
19: **end if**
20: $\mathbf{a}_i'^{(t)} \leftarrow \mathbf{h}_i'^{(t)} \circ \text{Act}'(\mathbf{a}_i^{(t)}) \triangleright \mathbf{a}_i'^{(t)} \in \mathbb{R}^h$
21: $\nabla \mathbf{W}_{h,i} \leftarrow \mathbf{a}_i'^{(t)} (\mathbf{h}_i^{(t-1)})^\top$
22: $\nabla \mathbf{W}_{e,i} \leftarrow \mathbf{a}_i'^{(t)} (\mathbf{x}_i^{(t)})^\top$
23: $\nabla b_{1,i} \leftarrow \mathbf{a}_i'^{(t)}$
24: **end for**

UPDATE WEIGHTS

25: $\mathbf{W}_{y,i} \leftarrow \mathbf{W}_{y,i-1} - \eta \cdot \frac{1}{T} \nabla \mathbf{W}_{y,i}$
26: $\mathbf{W}_{h,i} \leftarrow \mathbf{W}_{h,i-1} - \eta \cdot \frac{1}{T} \nabla \mathbf{W}_{h,i}$
27: $\mathbf{W}_{e,i} \leftarrow \mathbf{W}_{e,i-1} - \eta \cdot \frac{1}{T} \nabla \mathbf{W}_{e,i}$
28: $b_{1,i} \leftarrow b_{1,i-1} - \eta \cdot \frac{1}{T} \nabla b_{1,i}$
29: $b_{2,i} \leftarrow b_{2,i-1} - \eta \cdot \frac{1}{T} \nabla b_{2,i}$

A.2. Proofs, Arguments and Commitment

A.2.1. Zero-knowledge Arguments of Knowledge. An argument system for an NP relation $\mathcal{R}$ is a protocol that enables a computationally-bounded prover $\mathcal{P}$ to convince a verifier $\mathcal{V}$ that there exists a witness ω such that $(x, \omega) \in \mathcal{R}$ for some input x . If there exists an extractor $\mathcal{E}$ that extracts the witness ω from $\mathcal{P}$'s communication with $\mathcal{V}$, then we say that the argument is a zero-knowledge argument of knowledge. Let $\mathcal{G}$ denote the generator used to generate the public parameter pp .

Definition 3. Given an NP relation $\mathcal{R}$. A tuple of algorithm $(\mathcal{G}, \mathcal{P}, \mathcal{V})$ is a zero-knowledge argument of knowledge for $\mathcal{R}$ if the following conditions hold:

- **Correctness.** For any $\text{pp} \leftarrow \mathcal{G}(1^\lambda)$ and $(x, \omega) \in \mathcal{R}$,

$$\langle \mathcal{P}(\text{pp}, \omega), \mathcal{V}(\text{pp}) \rangle(x) = 1.$$

- **Knowledge Soundness.** For any PPT adversary $\mathcal{A}$, there exists a PPT extractor $\mathcal{E}$ such that given the access to $\mathcal{A}$'s messages and randomness

$$\Pr \left[\mathcal{V}(\text{pp}, x, \pi^*) = 1 \wedge \begin{array}{l} (x, \omega) \notin \mathcal{R} \\ \text{pp} \leftarrow \mathcal{G}(1^\lambda), \\ \pi^* \leftarrow \mathcal{A}(x, \text{pp}) \\ \omega \leftarrow \mathcal{E}(\text{pp}, x, \pi^*) \end{array} \right] \leq \text{negl}(\lambda)$$

- **Zero-knowledge.** There exists a PPT simulator $\mathcal{S}$ such that for all PPT adversary $\mathcal{A}$, auxiliary input $z \in \{0, 1\}^*$, $(x, \omega) \in \mathcal{R}$, and $\text{pp} \leftarrow \mathcal{G}(1^\lambda)$, it holds that

$$\text{View}(\langle \mathcal{P}(\text{pp}, \omega), \mathcal{A}(\text{pp}, z) \rangle(x)) \approx \mathcal{S}^{\mathcal{A}}(x, z)$$

A.2.2. Polynomial Commitment Schemes.

Definition 4 (Polynomial Commitment Schemes.). Given a finite field $\mathbb{F}$, a polynomial commitment scheme PCS is a tuple of PPT algorithms (KeyGen, Commit, Open, Verify) where:

- $\text{pp} \leftarrow \text{KeyGen}(1^\lambda)$
- $\sigma \leftarrow \text{Commit}(\text{pp}, f, r)$
- $(y, \pi) \leftarrow \text{Open}(\text{pp}, f, r, x)$
- $\{0, 1\} \leftarrow \text{Verify}(\text{pp}, \sigma, \pi, x, y)$

which satisfies the following properties:

- **Correctness.** For any $x \in \mathbb{F}$,

$$\Pr \left[\text{Verify}(\text{pp}, \sigma, \pi, x, y) = 1 \mid \begin{array}{l} \text{pp} \leftarrow \text{KeyGen}(1^\lambda), \\ \sigma \leftarrow \text{Commit}(\text{pp}, f, r) \\ (y, \pi) \leftarrow \text{Open}(\text{pp}, f, r, x) \end{array} \right] = 1$$

- **Knowledge Soundness.** For any PPT adversary $\mathcal{A}$, there exists a PPT extractor $\mathcal{E}$ such that given the access to $\mathcal{A}$'s messages and randomness, the following probability is $\text{negl}(\lambda)$

$$\Pr \left[\begin{array}{l} \text{Verify}(\text{pp}, \sigma, \pi, x, y) = 1 \\ \wedge \sigma \leftarrow \text{Commit}(\text{pp}, f, r) \\ \wedge f(x) \neq y \end{array} \mid \begin{array}{l} \text{pp} \leftarrow \text{KeyGen}(1^\lambda), \\ (x, y) \leftarrow \mathcal{A}(\text{pp}), \\ (\sigma, \pi) \leftarrow \mathcal{A}(\text{pp}), \\ f \leftarrow \mathcal{E}^{\mathcal{A}}(\text{pp}) \end{array} \right]$$

- **Binding.** For any PPT adversary $\mathcal{A}$, the following probability is $\text{negl}(\lambda)$

$$\Pr \left[\begin{array}{l} f \neq f' \\ \text{Commit}(\text{pp}, f, r) = \text{Commit}(\text{pp}, f', r') \end{array} \mid \begin{array}{l} \text{pp} \leftarrow \text{KeyGen}(1^\lambda), \\ (f, r) \leftarrow \mathcal{A}(\text{pp}), \\ (f', r') \leftarrow \mathcal{A}(\text{pp}) \end{array} \right]$$

- For any $\text{pp} \leftarrow \text{KeyGen}(1^\lambda)$ and for all f, f' , the following distributions are statistically indistinguishable:

$$\text{Commit}(\text{pp}, f, r) \approx \text{Commit}(\text{pp}, f', r')$$

A.2.3. Incrementally Verifiable Computation (IVC).

Definition 5 (IVC [12]). An incrementally verifiable computation (IVC) scheme is defined by PPT algorithms $(\mathcal{G}, \mathcal{P}, \mathcal{V})$ and deterministic $\mathcal{K}$ denoting the generator, the prover, the verifier, and the encoder respectively. An IVC scheme $(\mathcal{G}, \mathcal{K}, \mathcal{P}, \mathcal{V})$ satisfies completeness if for any PPT adversary $\mathcal{A}$, the following probability is 1.

$$\Pr \left[\mathcal{V}(\text{vk}, i, z_0, \pi_i) = 1 \mid \begin{array}{l} \text{pp} \leftarrow \mathcal{G}(1^\lambda), \\ F(i, z_0, z_i, z_{i-1}, \omega_{i-1}, \pi_{i-1}) \leftarrow \mathcal{A}(\text{pp}), \\ (\text{pk}, \text{vk}) \leftarrow \mathcal{K}(\text{pp}, F), \\ z_i = F(z_{i-1}, \omega_{i-1}), \\ \mathcal{V}(\text{vk}, i-1, z_0, z_{i-1}, \pi_{i-1}) \\ \pi_i \leftarrow \mathcal{P}(\text{pk}, i, z_0, z_i; z_{i-1}, \omega_{i-1}, \pi_{i-1}) \end{array} \right]$$

where F is a polynomial function. An IVC scheme satisfies knowledge-soundness if for any constant $n \in \mathbb{N}$, any PPT adversaries $\mathcal{P}^*$, there exists PPT extractor $\mathcal{E}$, such that for any input randomness ρ , the following probability is $\text{negl}(\lambda)$

$$\Pr \left[\begin{array}{c} z_n \neq z, \\ \mathcal{V}(\text{vk}, n, z_0, z, \pi) = 1 \end{array} \mid \begin{array}{c} \text{pp} \leftarrow \mathcal{G}(1^\lambda), \\ F(z_0, z, \pi_{i-1}) \leftarrow \mathcal{P}^*(\text{pp}, \rho), \\ (\text{pk}, \text{vk}) \leftarrow \mathcal{K}(\text{pp}, F), \\ (\omega_0, \dots, \omega_{n-1}) \leftarrow \mathcal{E}(\text{pp}, z_0, z; \rho) \\ z_i \leftarrow F(z_{i-1}, \omega_{i-1}) \forall i \in \{1, \dots, n\} \end{array} \right]$$

An IVC scheme satisfies succinctness if the size of the IVC proof π does not grow with the number of applications n .

A.3. Sumcheck Protocol

Theorem 3. [22] Let $\mathbb{F}$ be a finite field and let $g : \mathbb{F}^\ell \rightarrow \mathbb{F}$ be an ℓ -variate polynomial with degree at most d . Then Protocol A.3 is an interactive proof for the statement

$$T = \sum_{x \in \{0,1\}^\ell} g(x),$$

with soundness error at most $O(d\ell/|\mathbb{F}|)$. The protocol uses ℓ rounds. The verifier runs in $O(d\ell)$ field operations plus a single oracle evaluation of g at a random point $g(r_1, \dots, r_\ell)$. The prover time can be reduced to $O(2^\ell)$ using techniques of [31].

Protocol A.3. Sumcheck Protocol

- Round 1: $\mathcal{P}$ sends a univariate polynomial

$$g_1(x_1) := \sum_{b_2, \dots, b_\ell \in \{0,1\}} g(x_1, b_2, \dots, b_\ell).$$

$\mathcal{V}$ checks $g_1(0) + g_1(1) \stackrel{?}{=} T$ and sends a random challenge $r_1 \in \mathbb{F}$.

- Round i ($2 \leq i \leq \ell - 1$): $\mathcal{P}$ sends

$$g_i(x_i) := \sum_{b_{i+1}, \dots, b_\ell \in \{0,1\}} g(r_1, \dots, r_{i-1}, x_i, b_{i+1}, \dots, b_\ell).$$

$\mathcal{V}$ checks $g_{i-1}(r_{i-1}) \stackrel{?}{=} g_i(0) + g_i(1)$, then sends a random challenge $r_i \in \mathbb{F}$.

- Final round ($i = \ell$): $\mathcal{P}$ sends

$$g_\ell(x_\ell) := g(r_1, \dots, r_{\ell-1}, x_\ell),$$

and $\mathcal{V}$ checks $g_{\ell-1}(r_{\ell-1}) \stackrel{?}{=} g_\ell(0) + g_\ell(1)$. Finally, $\mathcal{V}$ samples $r_\ell \in \mathbb{F}$ and checks

$$g_\ell(r_\ell) \stackrel{?}{=} g(r_1, \dots, r_\ell),$$

using oracle access to g . $\mathcal{V}$ accepts if and only if all checks pass.

A.4. GKR Protocol

Theorem 4. [31] Let $\mathcal{C} \in \mathbb{F}^n \rightarrow \mathbb{F}^k$ be a layered arithmetic circuit with d layers. Protocol A.4 is an interactive proof for the function computed by $\mathcal{C}$ with soundness $O(d \log |\mathcal{C}|/|\mathbb{F}|)$. The protocol uses $O(d \log |\mathcal{C}|)$ rounds. The prover time is $O(|\mathcal{C}| \log |\mathcal{C}|)$. For log-space uniform circuits $T = \text{polylog}|\mathcal{C}|$. The verifier time is $O(n + k + d \log |\mathcal{C}| + T)$. The prover time can be reduced to $O(|\mathcal{C}|)$.

Appendix B. PBPTT Protocol

In this section, we present our complete PBPTT protocol of RNN in Protocol B.

We formally prove the security of Protocol B.

Proof of Theorem 1. Correctness. The correctness is straightforward by the correctness of GKR, $\text{Sum}_{\text{MatMul}}$, and LogUp protocols.

Knowledge Soundness. Suppose a malicious prover $\mathcal{P}^*$ produces an accepting transcript for iteration i . Consider Stage I, e.g. $\mathbf{W}_y$. If $\mathcal{P}^*$ provides an incorrect value $\mathbf{W}_{y,i}^* \neq \mathbf{W}_{y,i}$, then by the Schwartz-Zippel lemma and the soundness of GKR and $\text{Sum}_{\text{MatMul}}$, this will be detected with high probability unless either: (i) the corresponding input parameter $\mathbf{W}_{y,i-1}^*$ is incorrect, or (ii) the gradient contribution $\nabla_{\mathbf{W}_{y,i}}^*$ is incorrect. Case (ii) reduces to the backward-pass check: if $\nabla_{\mathbf{W}_{y,i}}^* \neq \nabla_{\mathbf{W}_{y,i}}$, then with high probability one of the underlying relations (e.g. $\delta^{(t)}(\mathbf{h}^{(t)})^\top$) will fail a $\text{Sum}_{\text{MatMul}}$ verification. Similarly, checking $\delta^{(t)} = \hat{\mathbf{y}}^{(t)} - \mathbf{y}^{(t)}$ guarantees that any incorrect value in $\delta^{(t)}$ propagates to a violation with respect to the committed lookup witnesses or parameters. By induction across stages, any incorrect value in intermediate traces (hidden states, activations, gradients) reduces to an inconsistency in the committed parameters (θ_{i-1}, θ_i) or in the lookup witnesses. At the end of Stage IV, all random evaluation claims are reduced to a constant number of PCS openings, whose binding property ensures uniqueness. Therefore, an extractor can invoke the extractor of PCS to recover the unique committed parameters and lookup witnesses, establishing knowledge soundness.

Zero-knowledge. The zero-knowledge property follows by invoking the simulators of each components to produce commitments, evaluation claims, and transcript that are indistinguishable from a real execution. $\square$

Appendix C. Construction of SUMMER

We formally prove the security of Protocol 6.2.5.

Proof of Theorem 2. Correctness. Suppose $\mathcal{P}$ honestly executes iteration i with inputs $(\theta_{i-1}, \mathcal{D})$. By construction, PBPTT.Prove produces valid transcripts for all linear relations via $\text{Sum}_{\text{MatMul}}$ and nonlinear relations via LogUp, and returns the correct updated parameters θ_i . Since the child proofs Π_{i-1} are already accepted, the augment function F_A folds only valid instances and Merkle-certified codewords. By induction, the verifier $\mathcal{V}$ will accept every Π_i , and at the recursion root, PCS.Verify ensures that the committed inputs/outputs are consistent. Thus, every honestly generated proof is accepted with probability 1.

Knowledge Soundness. We construct an extractor $\mathcal{E}$ in the Fiat-Shamir random oracle model.

By invoking the PCS extractor with Π_i^{open} , $\mathcal{E}$ extracts the opened values for the constant number of batched commitments at the root. By binding of PCS, these are uniquely determined. From Π_i^{agg} , $\mathcal{E}$ obtains the k child step transcripts that were batch verified *in-circuit* and the linear-combination coefficients $\{\lambda_j\}$ derived from the Fiat-Shamir transcript, and verifies that the parent's

Protocol A.4. GKR Protocol

Let $\mathbb{F}$ be a finite field. Let $\mathcal{C} : \mathbb{F}^n \rightarrow \mathbb{F}^k$ be a layered arithmetic circuit with d layers. A prover $\mathcal{P}$ wants to convince a verifier $\mathcal{V}$ that $\text{out} = \mathcal{C}(\text{in})$ where in is the input from $\mathcal{V}$ and out is the corresponding output computed by the circuit. WLOG, assume n and k are both powers of 2.

- 1) Define the multilinear extension of array out as $\tilde{V}_0$. $\mathcal{V}$ samples a random $g^{(0)} \in \mathbb{F}^{s_0}$ and sent it to $\mathcal{P}$. Both parties compute $\tilde{V}_0(g)$.
- 2) $\mathcal{P}$ and $\mathcal{V}$ run a sumcheck protocol on

$$\tilde{V}_0(g^{(0)}) = \sum_{x, y \in \{0,1\}^{s_1}} \text{mult}_1(g^{(0)}, x, y) \cdot \left(\tilde{V}_1(x) \tilde{V}_1(y) \right) + \text{add}_1(g^{(0)}, x, y) \cdot \left(\tilde{V}_1(x) + \tilde{V}_1(y) \right)$$

At the end of the sumcheck, $\mathcal{V}$ receives $\tilde{V}_1(u^{(1)})$ and $\tilde{V}_1(v^{(1)})$. $\mathcal{V}$ computes $\text{mult}_1(g^{(0)}, u^{(1)}, y^{(1)})$ and $\text{add}_1(g^{(0)}, u^{(1)}, y^{(1)})$. $\mathcal{V}$ checks that

$$\text{mult}_1(g^{(0)}, u^{(1)}, v^{(1)}) \tilde{V}_1(u^{(1)}) \tilde{V}_1(v^{(1)}) + \text{add}_1(g^{(0)}, u^{(1)}, v^{(1)}) \tilde{V}_1(u^{(1)}) + \tilde{V}_1(v^{(1)})$$

equals to the last message of the sumcheck.

- 3) for $i = 1, \dots, d-1$:

- $\mathcal{V}$ randomly samples $\alpha^{(i)}, \beta^{(i)} \in \mathbb{F}$ and sends them to $\mathcal{P}$.
- $\mathcal{P}$ and $\mathcal{V}$ run a sumcheck protocol on the equation:

$$\begin{aligned} \alpha^{(i)} \tilde{V}_i(u^{(i)}) + \beta^{(i)} \tilde{V}_i(v^{(i)}) &= \sum_{x, y \in \{0,1\}^{s_{i+1}}} ((\alpha^{(i)} \text{mult}_{i+1}(u^{(i)}, x, y) + \beta^{(i)} \text{mult}_{i+1}(v^{(i)}, x, y)) \cdot \tilde{V}_{i+1}(x) \tilde{V}_{i+1}(y) \\ &\quad + (\alpha^{(i)} \text{add}_{i+1}(u^{(i)}, x, y) + \beta^{(i)} \text{add}_{i+1}(v^{(i)}, x, y)) \cdot (\tilde{V}_{i+1}(x) + \tilde{V}_{i+1}(y))) \end{aligned}$$

- At the end of the sumcheck, $\mathcal{P}$ sends to $\mathcal{V}$ $\tilde{V}_{i+1}(u^{(i+1)})$ and $\tilde{V}_{i+1}(v^{(i+1)})$.
- $\mathcal{V}$ computes $\text{Mult}_{i+1}(x) = \text{mult}_{i+1}(x, u^{(i+1)}, v^{(i+1)})$ and $\text{Add}_{i+1}(x) = \text{add}_{i+1}(x, u^{(i+1)}, v^{(i+1)})$. $\mathcal{V}$ checks that if the following equals to the last message of the sum check

$$\begin{aligned} &(\alpha^{(i)} \text{Mult}_{i+1}(u^{(i)}) + \beta^{(i)} \text{Mult}_{i+1}(v^{(i)})) \cdot \tilde{V}_{i+1}(u^{(i+1)}) \tilde{V}_{i+1}(v^{(i+1)}) \\ &+ (\alpha^{(i)} \text{Add}_{i+1}(u^{(i)}) + \beta^{(i)} \text{Add}_{i+1}(v^{(i)})) \cdot (\tilde{V}_{i+1}(u^{(i+1)}) + \tilde{V}_{i+1}(v^{(i+1)})) \end{aligned}$$

- If the check passes, $\mathcal{V}$ uses $\tilde{V}_{i+1}(u^{(i+1)})$ and $\tilde{V}_{i+1}(v^{(i+1)})$ as new claims and proceeds to the next layer. Otherwise, $\mathcal{V}$ aborts and rejects.

- 4) At the input layer d , $\mathcal{V}$ has two claims: $\tilde{V}_d(u^{(d)})$ and $\tilde{V}_d(v^{(d)})$. $\mathcal{V}$ queries the oracle of evaluations of $\tilde{V}_d$ at both $u^{(d)}$ and $v^{(d)}$ and checks whether the evaluations match the claimed values. If both checks succeed, $\mathcal{V}$ accepts; otherwise, it rejects.

value/evaluation claims are the linear combination of children and that the aggregated codeword matches Merkle samples on a public random set. By the Merkle sampling check and Schwartz–Zippel lemma, any incorrect equality is rejected with high probability; thus $\mathcal{E}$ obtains the next accumulator and the k child instances. For each child step, $\mathcal{E}$ invokes the extractor of PBPTT to obtain the per-iteration committed objects: the parameter commitments $(\sigma_{\theta_{i-1}}, \sigma_{\theta_i})$ and the per-iteration lookup witness commitment $\sigma_{Q,i}$. Any incorrect parameter, gradient, or intermediate value is already detected with high probability by PBPTT's soundness. Applying (ii)–(iii) recursively over the recursion tree reaches all leaves. The root's constant number of PCS openings from (i) binds the concrete values needed to certify all transcript-bound claims. By binding and extractability of PCS, the extracted parameters and lookup witnesses are unique and consistent.

Zero-knowledge. The zero-knowledge property follows from each component: PCS (and Merkle) are hiding; PBPTT admits a simulator; the augment function F_A adds only randomized linear combinations and simulatable Merkle checks; the root invokes the PCS simulator for batched openings. The simulated transcript is indistinguishable from a real proof. $\square$

Protocol B. Proof of Gradient Descent of RNN Protocol

Parameters: Let GKR be a generic GKR-style zero-knowledge proof system, $\text{Sum}_{\text{MatMul}}$ be the time-optimal sumcheck protocol for matrix multiplication [28]. Let LogUp be a logarithmic-derivative TaSSE-style lookup argument. Throughout the protocol, every linear relation listed under *Linear Operations (Section 3.2)* (matrix-matrix/vector products, Hadamard product, and summations) is reduced to a single matrix multiplication instance and proved by invoking $\text{Sum}_{\text{MatMul}}$. Let $\text{PCS} = (\text{KeyGen}, \text{Commit}, \text{Open}, \text{Verify})$ be the multivariate polynomial commitment scheme with public parameter pp . Let the training window have T time steps.

Initialization:

- 1) On input sequence $\{\mathbf{x}^{(t)}\}_{t=1}^T$ with labels $\{\mathbf{y}^{(t)}\}_{t=1}^T$, initial state $\mathbf{h}^{(0)}$, and parameters $(\mathbf{W}_{x,i-1}, \mathbf{W}_{h,i-1}, \mathbf{W}_{y,i-1}, \mathbf{b}_{1,i-1}, \mathbf{b}_{2,i-1})$, the prover $\mathcal{P}$ runs BPTT (Algorithm 1) to obtain forward traces $\{\mathbf{a}^{(t)}, \mathbf{h}^{(t)}, \mathbf{z}^{(t)}, \hat{\mathbf{y}}^{(t)}\}$, backward traces $\{\delta^{(t)}, \mathbf{h}'^{(t)}, \mathbf{a}'^{(t)}\}$, the gradients $(\nabla_{\mathbf{W}_{x,i}}, \nabla_{\mathbf{W}_{h,i}}, \nabla_{\mathbf{W}_{y,i}}, \nabla_{\mathbf{b}_{1,i}}, \nabla_{\mathbf{b}_{2,i}})$, and updated parameters $(\mathbf{W}_{x,i}, \mathbf{W}_{h,i}, \mathbf{W}_{y,i}, \mathbf{b}_{1,i}, \mathbf{b}_{2,i})$.
- 2) $\mathcal{P}$ concatenates the initial parameters and the updated parameters, and runs $\sigma_{\theta,i-1} \leftarrow \text{PCS.Commit}(\text{concat}(\mathbf{W}_{x,i-1}, \mathbf{W}_{h,i-1}, \mathbf{W}_{y,i-1}, \mathbf{b}_{1,i-1}, \mathbf{b}_{2,i-1}))$, $\sigma_{\theta,i} \leftarrow \text{PCS.Commit}(\text{concat}(\mathbf{W}_{x,i}, \mathbf{W}_{h,i}, \mathbf{W}_{y,i}, \mathbf{b}_{1,i}, \mathbf{b}_{2,i}))$. $\mathcal{P}$ sends $\sigma_{\theta,i-1}, \sigma_{\theta,i}$ to the verifier $\mathcal{V}$.
- 3) $\mathcal{P}$ runs $\sigma_{T,\text{act}} \leftarrow \text{PCS.Commit}(\widetilde{T}_{\text{act}})$, $\sigma_{T,\text{sm}} \leftarrow \text{PCS.Commit}(\widetilde{T}_{\text{sm}})$ once (in the first iteration) to the precomputed lookup tables and runs $\sigma_{Q,i} \leftarrow \text{PCS.Commit}(\widetilde{Q}_i)$ to the per-round lookup queries (indices and any auxiliary values) packed as a vector Q_i . $\mathcal{P}$ sends $\sigma_{T,\text{act}}, \sigma_{T,\text{sm}}, \sigma_{Q,i}$ to $\mathcal{V}$.

Stage I: Proof of Update.

- 1) $\mathcal{V}$ samples r_{out} and sends it to $\mathcal{P}$.
- 2) $\mathcal{P}$ evaluates $\widetilde{\mathbf{W}}_{y,i}(r_{\text{out}})$, $\widetilde{\mathbf{W}}_{h,i}(r_{\text{out}})$, $\widetilde{\mathbf{W}}_{x,i}(r_{\text{out}})$, $\widetilde{\mathbf{b}}_{1,i}(r_{\text{out}})$, $\widetilde{\mathbf{b}}_{2,i}(r_{\text{out}})$ and sends these values to $\mathcal{V}$.
- 3) $\mathcal{V}$ and $\mathcal{P}$ run GKR on:

$$\begin{aligned} \mathbf{W}_{y,i} &= \mathbf{W}_{y,i-1} - \eta \cdot \nabla_{\mathbf{W}_{y,i}} & \mathbf{W}_{h,i} &= \mathbf{W}_{h,i-1} - \eta \cdot \nabla_{\mathbf{W}_{h,i}} & \mathbf{W}_{x,i} &= \mathbf{W}_{x,i-1} - \eta \cdot \nabla_{\mathbf{W}_{x,i}} \\ \mathbf{b}_{1,i} &= \mathbf{b}_{1,i-1} - \eta \cdot \nabla_{\mathbf{b}_{1,i}} & \mathbf{b}_{2,i} &= \mathbf{b}_{2,i-1} - \eta \cdot \nabla_{\mathbf{b}_{2,i}} \end{aligned}$$

At the end of this protocol, $\mathcal{V}$ receives the evaluations of $(\widetilde{\mathbf{W}}_{*,i-1}(r_{1,\mathbf{W}_*}), \widetilde{\nabla_{\mathbf{W}_{*,i}}}(r_{1,\nabla_{\mathbf{W}_*}}))$ and $(\widetilde{\mathbf{b}}_{*,i-1}(r_{1,\mathbf{b}_*}), \widetilde{\nabla_{\mathbf{b}_{*,i}}}(r_{1,\nabla_{\mathbf{b}_*}}))$.

Stage II: Proof of Backward Pass.

- 1) With $\widetilde{\nabla_{\mathbf{W}_{x,i}}}(r_{1,\nabla_{\mathbf{W}_{x,i}}})$ from Stage I, $\mathcal{V}$ and $\mathcal{P}$ run $\text{Sum}_{\text{MatMul}}$ on

$$\nabla_{\mathbf{W}_{x,i}} = \frac{1}{T} \sum_{t=1}^T \mathbf{a}'^{(t)} \cdot (\mathbf{x}^{(t)})^\top$$

At the end of the protocol, $\mathcal{V}$ receives evaluations of $\widetilde{\mathbf{a}'^{(t)}}(r_{2,\mathbf{a}'^{(t)}}^{(1)})$ and $\widetilde{\mathbf{x}^{(t)}}(r_{2,\mathbf{x}^{(t)}})$ for all $t \in [1, T]$.

- 2) With $\widetilde{\nabla_{\mathbf{W}_{h,i}}}(r_{1,\nabla_{\mathbf{W}_{h,i}}})$ from Stage I, $\mathcal{V}$ and $\mathcal{P}$ run $\text{Sum}_{\text{MatMul}}$ on

$$\nabla_{\mathbf{W}_{h,i}} = \frac{1}{T} \sum_{t=1}^T \mathbf{a}'^{(t)} \cdot (\mathbf{h}^{(t-1)})^\top$$

At the end of the protocol, $\mathcal{V}$ receives evaluations of $\widetilde{\mathbf{a}'^{(t)}}(r_{2,\mathbf{a}'^{(t)}}^{(2)})$ and $\widetilde{\mathbf{h}^{(t-1)}}(r_{2,\mathbf{h}^{(t-1)}}^{(1)})$ for all $t \in [1, T]$.

- 3) With $\widetilde{\nabla_{\mathbf{b}_{1,i}}}(r_{1,\nabla_{\mathbf{b}_{1,i}}})$ from Stage I, $\mathcal{V}$ and $\mathcal{P}$ run $\text{Sum}_{\text{MatMul}}$ on

$$\nabla_{\mathbf{b}_{1,i}} = \frac{1}{T} \sum_{t=1}^T \mathbf{a}_i'^{(t)}$$

At the end of the protocol, $\mathcal{V}$ receives evaluations of $\widetilde{\mathbf{a}'^{(t)}}(r_{2,\mathbf{a}'^{(t)}}^{(3)})$ for all $t \in [1, T]$.

- 4) $\mathcal{V}$ and $\mathcal{P}$ combine evaluations $\widetilde{\mathbf{a}'^{(t)}}(r_{2,\mathbf{a}'^{(t)}}^{(1)})$, $\widetilde{\mathbf{a}'^{(t)}}(r_{2,\mathbf{a}'^{(t)}}^{(2)})$, and $\widetilde{\mathbf{a}'^{(t)}}(r_{2,\mathbf{a}'^{(t)}}^{(3)})$ for all $t \in [1, T]$ by running an evaluation reduction sumcheck. Have the combination, along with the LogUp proof against the committed Act table, $\mathcal{V}$ and $\mathcal{P}$ prove

$$\mathbf{a}'^{(t)} = \mathbf{h}^{(t)} \circ \text{Act}'(\mathbf{a}^{(t)}) \quad (\text{e.g., } \tanh' : 1 - \mathbf{h}^{(t)} \circ \mathbf{h}^{(t)}).$$

At the end of the protocol, $\mathcal{V}$ receives evaluations of $\widetilde{\mathbf{h}'^{(t)}}(r_{2,\mathbf{h}'^{(t)}}^{(2)})$ for all $t \in [1, T]$.

- 5) With $\widetilde{\mathbf{h}'^{(t)}}(r_{2,\mathbf{h}'^{(t)}}^{(2)})$, for $t = T$, $\mathcal{V}$ and $\mathcal{P}$ run $\text{Sum}_{\text{MatMul}}$ on

$$\mathbf{h}'^{(t)} = \mathbf{W}_{y,i-1}^\top \cdot \delta^{(t)}$$

At the end of the protocol, $\mathcal{V}$ receives evaluations of $\widetilde{\mathbf{W}}_{y,i-1}(r_{2,\mathbf{W}_{y,i-1}})$ and $\widetilde{\delta^{(t)}}(r_{2,\delta^{(t)}}^{(1)})$ for all $t \in [1, T]$.

For $t \neq T$, $\mathcal{V}$ and $\mathcal{P}$ run $\text{Sum}_{\text{MatMul}}$ on

$$\mathbf{h}'^{(t)} = \mathbf{W}_{y,i-1}^\top \cdot \delta^{(t)} + \mathbf{W}_{h,i-1}^\top \cdot \mathbf{a}'^{(t+1)}$$

At the end of the protocol, $\mathcal{V}$ receives evaluations of $\widetilde{\mathbf{W}}_{y,i-1}(r_{2,\mathbf{w}_{y,i-1}})$, $\widetilde{\mathbf{W}}_{h,i-1}(r_{2,\mathbf{w}_{h,i-1}})$, $\widetilde{\mathbf{a}^{(t+1)}}(r_{2,\mathbf{a}^{(t+1)}}^{(4)})$, and $\widetilde{\delta^{(t)}}(r_{2,\delta^{(t)}}^{(1)})$ for all $t \in [1, T]$.

- 6) With $\widetilde{\nabla_{\mathbf{b}_{2,i}}}(r_{1,\nabla_{\mathbf{b}_{2,i}}})$ from Stage I, $\mathcal{V}$ and $\mathcal{P}$ run $\text{Sum}_{\text{MatMul}}$ on

$$\nabla_{\mathbf{b}_{2,i}} = \frac{1}{T} \sum_{t=1}^T \delta^{(t)}$$

At the end of the protocol, $\mathcal{V}$ receives evaluations of $\widetilde{\delta^{(t)}}(r_{2,\delta^{(t)}}^{(2)})$ for all $t \in [1, T]$.

- 7) With $\widetilde{\nabla_{\mathbf{w}_{y,i}}}(r_{1,\nabla_{\mathbf{w}_{y,i}}})$ from Stage I, $\mathcal{V}$ and $\mathcal{P}$ run $\text{Sum}_{\text{MatMul}}$ on

$$\nabla_{\mathbf{w}_{y,i}} = \frac{1}{T} \sum_{t=1}^T \delta^{(t)} \cdot (\mathbf{h}^{(t)})^\top$$

At the end of the protocol, $\mathcal{V}$ receives evaluations of $\widetilde{\delta^{(t)}}(r_{2,\delta^{(t)}}^{(3)})$ and $\widetilde{\mathbf{h}^{(t)}}(r_{2,\mathbf{h}^{(t)}}^{(2)})$ for all $t \in [1, T]$.

- 8) $\mathcal{V}$ and $\mathcal{P}$ combine evaluations $\widetilde{\delta^{(t)}}(r_{2,\delta^{(t)}}^{(1)})$, $\widetilde{\delta^{(t)}}(r_{2,\delta^{(t)}}^{(2)})$, and $\widetilde{\delta^{(t)}}(r_{2,\delta^{(t)}}^{(3)})$ for all $t \in [1, T]$ by running an evaluation reduction sumcheck. Having the combination, $\mathcal{V}$ and $\mathcal{P}$ run GKR on

$$\delta_i^{(t)} = \hat{\mathbf{y}}^{(t)} - \mathbf{y}^{(t)}$$

At the end of the protocol, $\mathcal{V}$ receives evaluations of $\widetilde{\hat{\mathbf{y}}^{(t)}}(r_{2,\hat{\mathbf{y}}^{(t)}})$ and $\widetilde{\mathbf{y}^{(t)}}(r_{2,\mathbf{y}^{(t)}})$ for all $t \in [1, T]$.

Stage III: Proof of Forward Pass.

- 1) With $\widetilde{\hat{\mathbf{y}}^{(t)}}(r_{2,\hat{\mathbf{y}}^{(t)}})$ from Stage II, along with the LogUp proof against the committed softmax table, $\mathcal{V}$ and $\mathcal{P}$ prove

$$\hat{\mathbf{y}}^{(t)} = \text{Softmax}(\mathbf{z}^{(t)})$$

At the end of the protocol, $\mathcal{V}$ receives evaluations of $\widetilde{\mathbf{z}^{(t)}}(r_{3,\mathbf{z}^{(t)}})$ for all $t \in [1, T]$.

- 2) With $\widetilde{\mathbf{z}^{(t)}}(r_{3,\mathbf{z}^{(t)}})$, $\mathcal{V}$ and $\mathcal{P}$ run $\text{Sum}_{\text{MatMul}}$ on

$$\mathbf{z}^{(t)} \leftarrow \mathbf{W}_{y,i-1} \cdot \mathbf{h}^{(t)} + \mathbf{b}_{2,i-1}$$

At the end of the protocol, $\mathcal{V}$ receives evaluations of $\widetilde{\mathbf{W}}_{y,i-1}(r_{3,\mathbf{w}_{y,i-1}})$, $\widetilde{\mathbf{h}^{(t)}}(r_{3,\mathbf{h}^{(t)}})$ for all $t \in [1, T]$, and $\widetilde{\mathbf{b}_{2,i-1}}(r_{3,\mathbf{b}_{2,i-1}})$.

- 3) With $\widetilde{\mathbf{h}^{(t)}}(r_{3,\mathbf{h}^{(t)}})$, along with the LogUp proof against the committed Act table, $\mathcal{V}$ and $\mathcal{P}$ prove

$$\mathbf{h}^{(t)} = \text{Act}(\mathbf{a}^{(t)})$$

At the end of the protocol, $\mathcal{V}$ receives evaluations of $\widetilde{\mathbf{a}^{(t)}}(r_{3,\mathbf{a}^{(t)}})$ for all $t \in [1, T]$.

- 4) With $\widetilde{\mathbf{a}^{(t)}}(r_{3,\mathbf{a}^{(t)}})$, $\mathcal{V}$ and $\mathcal{P}$ run $\text{Sum}_{\text{MatMul}}$ on

$$\mathbf{a}^{(t)} = \mathbf{W}_{h,i-1} \cdot \mathbf{h}^{(t-1)} + \mathbf{W}_{x,i-1} \cdot \mathbf{x}^{(t)} + \mathbf{b}_{1,i-1}$$

At the end of the protocol, $\mathcal{V}$ receives evaluations of $\widetilde{\mathbf{W}}_{h,i-1}(r_{3,\mathbf{w}_{h,i-1}})$, $\widetilde{\mathbf{W}}_{x,i-1}(r_{3,\mathbf{w}_{x,i-1}})$, $\widetilde{\mathbf{b}_{1,i-1}}(r_{3,\mathbf{b}_{1,i-1}})$, $\widetilde{\mathbf{h}^{(t-1)}}(r'_{3,\mathbf{h}^{(t-1)}})$, $\widetilde{\mathbf{x}^{(t)}}(r_{3,\mathbf{x}^{(t)}})$ for all $t \in [1, T]$.

Stage IV: Proof of evaluation reduction.

- 1) $\mathcal{V}$ and $\mathcal{P}$ combine evaluations of form $\widetilde{\mathbf{W}}_{*,i-1}(r_{1,\mathbf{w}_{*,i-1}})$, $\widetilde{\mathbf{W}}_{*,i-1}(r_{2,\mathbf{w}_{*,i-1}})$, $\widetilde{\mathbf{W}}_{*,i-1}(r_{3,\mathbf{w}_{*,i-1}})$ into a single evaluation $\widetilde{\mathbf{W}}_{*,i-1}(r_{\text{in},\mathbf{w}_{*,i-1}})$, evaluations of form $\widetilde{\mathbf{b}_{*,i-1}}(r_{1,\mathbf{b}_{*,i-1}})$, $\widetilde{\mathbf{b}_{*,i-1}}(r_{2,\mathbf{b}_{*,i-1}})$, $\widetilde{\mathbf{b}_{*,i-1}}(r_{3,\mathbf{b}_{*,i-1}})$ into a single evaluation $\widetilde{\mathbf{b}_{*,i-1}}(r_{\text{in},\mathbf{b}_{*,i-1}})$, evaluations of form $\widetilde{\mathbf{x}^{(t)}}(r_{2,\mathbf{x}^{(t)}})$ and $\widetilde{\mathbf{x}^{(t)}}(r_{3,\mathbf{x}^{(t)}})$ for all $t \in [1, T]$ into a single evaluation $\widetilde{\mathbf{x}}(r_{\text{in},\mathbf{x}})$, evaluations of $\widetilde{\mathbf{y}^{(t)}}(r_{2,\mathbf{y}^{(t)}})$ for all $t \in [1, T]$ into a single evaluations $\widetilde{\mathbf{y}}(r_{\text{in},\mathbf{y}})$.
- 2) The combined evaluations can be verified by using the evaluation opening algorithm of commitment scheme PCS.Open($\mathbf{W}_{*,i-1}$), PCS.Open($\mathbf{W}_{*,i}$), PCS.Open($\mathbf{b}_{*,i-1}$), PCS.Open($\mathbf{b}_{*,i}$), PCS.Open($\mathbf{x}$), PCS.Open($\mathbf{y}$).